



ClansCore

Weiterentwicklung Discord-Bot für Vereine

Studien- und Bachelorarbeit im
Herbst-Semester 2025

Ausdruck:

17.10.2025

Team / Autoren:

Vanessa Alves und Joel Thoma

Referent:

Prof. Dr. Frieder Loch

Koreferent:

Julia Czerniak-Wilmes

Gegenleser:

Severin Dellsperger

Inhaltsverzeichnis

I	Produkt Dokumentation	1
1	Einleitung	1
1.1	Problemstellung	1
1.2	Vorarbeiten	1
1.3	Ziel der Arbeit und Abgrenzung	2
1.4	Rahmenbedingungen	2
1.5	Stand der Technik	2
2	Anforderungen	4
2.1	Priorisierung der Anforderungen	4
2.2	Legende zum Erfüllungsgrad der Anforderungen	4
2.3	User Stories	4
2.4	Functional Requirements	5
2.5	Use Cases	7
2.6	Non-Functional Requirements	11
3	Erweitertes Gamification-Konzept	15
3.1	Interviews zur Motivation von Vereinsmitgliedern	15
3.1.1	Methodisches Vorgehen	15
3.1.2	Interviewleitfaden	16
3.1.3	Mehrwert gegenüber dem Vorprojekt	16
3.2	Octalysis Framework	16
3.2.1	ClansCore im Octalysis Framework	17
4	Erweitertes Sicherheitskonzept	19
4.1	Ziel und Geltungsbereich	19
4.2	Sicherheitsprinzipien	19
4.3	Authentifizierung und Autorisierung	20
4.3.1	Dienstkonto und automatisierte Zugriffe	20
4.3.2	Absicherung des Datenbankzugriffs	20
4.3.2.1	Bewertung und Entscheid	21
4.3.2.2	Übertragbarkeit auf andere Vereine	22
4.3.3	Dashboard-Login	22
4.4	Daten- und Schnittstellensicherheit	22
4.4.1	Eingabevalidierung	22
4.4.2	Synchronisierung & Konfliktlösung	23
4.5	Datenschutz und DSGVO/DSG-Umsetzung	23
4.6	Server- und Deployment-Sicherheit	23
4.7	Manipulations- und Integritätsschutz	24
4.8	Risikoanalyse und Massnahmen	24
4.9	Monitoring und Wartung	24
5	Systemanalyse, Wireframes und Architektur	25
5.1	Wireframes	25
6	Qualitätssicherung	27
7	Implementation	28
8	Fazit	29
II	Projekt Dokumentation	30
9	Projekt Plan	30
9.1	Zusammenarbeit	30
9.2	Meetings	30
9.3	Minimum Viable Product (MVP)	30
9.4	Long-Term Plan	31
9.4.1	Inception (Initiierung)	32
9.4.2	Elaboration (Ausarbeitung)	32
9.4.3	Construction (Konstruktion)	32

9.4.4	Transition (Übergang)	32
9.4.5	Presentation (Präsentation)	32
9.4.6	Meilensteine	32
9.5	Risikomanagement	33
9.5.1	Ausweichplan	36
10	Arbeitszeitrachweis	37
11	Tooling und Technologie-Entscheidungen	38
11.1	Jira	38
11.2	Teams	38
11.3	GitHub	38
11.3.1	GitHub Guidelines	39
11.4	Dokumentation	39
11.4.1	Dokumentation Guidelines	39
11.4.2	Dokumentation Deployment	39
11.5	Code und Entwicklung	40
11.5.1	Code Guidelines	40
11.5.2	Setup	40
11.5.3	Hosting	40
11.5.4	Code Deployment	40
11.6	Programmiersprachen und Frameworks	41
11.7	Datenbank	42
11.8	KI-Tools	42
12	Sitzungsprotokolle	43
13	Persönliche Erfahrungsberichte	46
13.1	Bericht Joel Thoma	46
13.2	Bericht Vanessa Alves	46
III	Glossar und Abkürzungsverzeichnis	47
IV	Literaturverzeichnis	49
V	Abbildungsverzeichnis	51
VI	Tabellenverzeichnis	52
VII	Code-Listings	55
VIII	Anhang	56

Kapitel I

Produkt Dokumentation

1 Einleitung

Digitale Kommunikationsplattformen wie Discord haben sich in den vergangenen Jahren zunehmend als zentrale Werkzeuge für die Organisation und Interaktion innerhalb von Online-Communities und Vereinen etabliert. Insbesondere im Bereich der Freizeit- und Gaming-Vereine bieten sie die Möglichkeit, Kommunikation, Eventplanung und Mitgliederverwaltung zu bündeln. Trotz dieser Vorteile stossen bestehende Plattformfunktionen bei wachsender Mitgliederzahl, zunehmender organisatorischer Komplexität und steigenden Anforderungen an Datensicherheit und Automatisierung an ihre Grenzen.

Vor diesem Hintergrund befasst sich die vorliegende Bachelorarbeit mit der Weiterentwicklung eines im Vorprojekt erstellten Discord-Bots, der vereinsrelevante Prozesse automatisiert und Gamification-Mechanismen zur Steigerung des Engagements nutzt. Ziel der Arbeit ist es, das bestehende System, um ein plattformunabhängiges Admin-Dashboard zu erweitern, welches die sichere, benutzerfreundliche und rollenbasierte Verwaltung von Mitgliederdaten und Gamification-Statistiken ermöglicht. Damit wird ein Beitrag zur Entkopplung der Vereinsverwaltung von der Kommunikationsplattform Discord geleistet und die Grundlage für eine modulare, erweiterbare Systemarchitektur geschaffen.

1.1 Problemstellung

Das im Vorprojekt entwickelte System ermöglicht die Automatisierung einiger zentraler Vereinsprozesse, ist jedoch ausschliesslich innerhalb der Discord-Umgebung funktionsfähig. Dadurch ist die Nutzung stark Discord-zentriert und für Personen ohne Discord-Konto oder mit eingeschränkter technischer Erfahrung nur begrenzt zugänglich.

Die Bearbeitung vereinsrelevanter Daten erfolgt derzeit ausschliesslich über direkten Zugriff auf die MongoDB-Datenbank. Dies erfordert spezifische technische Kenntnisse, birgt Sicherheitsrisiken und verhindert eine kontrollierte, benutzerfreundliche Datenverwaltung. Zudem existiert keine zentrale Oberfläche, die eine visuelle und statistisch fundierte Auswertung des Vereinsengagements ermöglicht.

Darüber hinaus erschwert die aktuelle Architektur die Anbindung zusätzlicher Plattformen oder externer Systeme, etwa zur Erweiterung um Web- oder Mobile-Anwendungen. Diese Einschränkungen verdeutlichen den Bedarf nach einer modularen, skalierbaren und plattformunabhängigen Systemlösung, die sowohl technische als auch organisatorische Herausforderungen adressiert.

1.2 Vorarbeiten

Das Vorprojekt „Discord-Bot für Vereine“ (Alves & Schnider, 2025) bildete die Grundlage für die vorliegende Arbeit. Es verfolgte das Ziel, zentrale Vereinsprozesse wie Mitgliederverwaltung, Eventorganisation und Gamification in einem Discord-Bot zu integrieren. Der entwickelte Prototyp implementierte ein Rollen- und Berechtigungssystem, eine Anbindung an die Google Calendar API sowie ein auf wissenschaftlichen Erkenntnissen basierendes Gamification-Konzept.

Während die Machbarkeit und der Nutzen eines solchen Systems bestätigt werden konnten, blieb dessen Erweiterbarkeit auf andere Plattformen sowie die administrative Steuerung offen. Das Admin-Dashboard wurde im Vorprojekt lediglich als potenzielle Weiterentwicklung beschrieben, jedoch nicht realisiert. Ebenso fehlten Mechanismen zur Messung und Visualisierung des Engagements sowie eine Entkopplung der Discord-spezifischen Logik von der Datenbankstruktur.

Die vorliegende Arbeit schliesst an diese Ergebnisse an und adressiert die genannten Defizite durch eine Neugestaltung der Architektur, eine verbesserte Datenkonsistenz zwischen Discord, Dashboard und Datenbank sowie durch die Integration einer administrativen Benutzeroberfläche. Gleichzeitig werden Erkenntnisse aus Interviews mit Präsidenten anderer Vereine in die konzeptionelle Weiterentwicklung des Gamification-Systems einbezogen, um dessen Übertragbarkeit und Nachhaltigkeit zu erhöhen.

1.3 Ziel der Arbeit und Abgrenzung

Ziel dieser Bachelorarbeit ist die Konzeption und Entwicklung einer plattformunabhängigen, administrativen Erweiterung des bestehenden Discord-Bots, die eine effiziente und sichere Verwaltung vereinsrelevanter Daten ermöglicht. Hierbei liegt der Fokus auf der funktionalen Umsetzung eines Admin-Dashboards sowie auf der architektonischen Modularisierung des Systems, um künftige Integrationen weiterer Plattformen leichter zu ermöglichen. Die neue Softwarelösung wird mit dem Namen ClansCore adressiert.

Im Einzelnen verfolgt die Arbeit folgende Teilziele:

- Entwurf und Implementierung eines funktionsfähigen Admin-Dashboards in ClansCore, zur Verwaltung von Mitgliedern, Rollen, Events und Gamification-Daten
- Refaktorisierung der ClansCore-Backend-Architektur zur Entkopplung von Discord-spezifischer Logik und zur Vorbereitung auf eine plattformübergreifende Erweiterung
- Entwicklung einer sicheren Schnittstellenkommunikation zwischen Bot, Dashboard und Datenbank zur Gewährleistung konsistenter Datenzustände innerhalb des gesamten ClansCore-Systems
- konzeptionelle und technische Erweiterung des Gamification-Systems, insbesondere zur Messung des Mitgliederengagements
- Evaluation von ClansCore durch Usability-Tests mit Vereinsmitgliedern

Diese Arbeit unterscheidet sich vom Vorprojekt durch ihren erweiterten technischen und analytischen Anspruch. Während das Vorprojekt primär die Funktionalität innerhalb von Discord validierte, liegt der Schwerpunkt der Bachelorarbeit auf der systematischen Generalisierung und Professionalisierung der Lösung als Gesamt-System namens ClansCore. Damit wird die Grundlage geschaffen, um das System langfristig als universelles Vereinsmanagement-Tool einsetzen zu können.

1.4 Rahmenbedingungen

Die Arbeit wurde im Rahmen der Bachelor- und Studienarbeit des Studiengangs Informatik an der Ostschweizer Fachhochschule (OST) durchgeführt.

- **Arbeitstyp:** Bachelorarbeit (360 Stunden) + Studienarbeit (240 Stunden)
- **Gesamtarbeitsaufwand:** 600 Stunden
- **ECTS-Punkte:** 12 ECTS (Bachelorarbeit) und 8 ECTS (Studienarbeit)

Der Schwerpunkt liegt auf der Konzeption, Entwicklung und Evaluation eines funktionalen Software-Systems mit wissenschaftlich begründetem Gamification-Ansatz, welches den Namen ClansCore trägt. Die Arbeit vereint methodische Ansätze aus den Bereichen Software-Engineering, Usability und Motivationsforschung und ist als entwicklungsorientierte empirische Arbeit mit prototypischem Charakter einzuordnen.

1.5 Stand der Technik

Der Stand der Technik im Bereich digitaler Vereinsorganisation wurde im Vorprojekt umfassend untersucht und bildet die Grundlage der vorliegenden Arbeit. Die damalige Marktanalyse zeigte, dass bestehende Systeme entweder auf die Interaktion und Gamification innerhalb von Discord oder auf die administrative Vereinsverwaltung über klassische Softwarelösungen fokussieren, jedoch keine integrative Verbindung beider Ansätze bieten.

Seit Abschluss des Vorprojekts im Juli 2025 sind zwar Weiterentwicklungen einzelner Systeme erkennbar, eine Lösung, welche Gamification, Datenverwaltung und plattformübergreifende Nutzung in einem kohärenten Konzept vereint, wurde bisher jedoch nicht öffentlich dokumentiert.

Im Bereich der Discord-Bots zählt MEE6 zu den meistgenutzten Anwendungen und bietet über das Plugin „Statistics Channels“ grundlegende Auswertungen zur Serveraktivität sowie ein Levelsystem zur Förderung der Nutzerinteraktion (MEE6-Contributors, 2025). Auch der Bot Arcane integriert vergleichbare Gamification-Funktionen in Form von Belohnungssystemen (Arcane-Contributors, 2025). Beide Systeme bleiben jedoch auf die Discord-Plattform beschränkt und verfügen über keine dokumentierten Schnittstellen zu externen Administrations- oder Datenverwaltungssystemen.

Demgegenüber adressieren klassische Vereinsmanagement-Lösungen wie ClubDesk oder WildApricot die organisatorischen Anforderungen von Vereinen, insbesondere in den Bereichen Mitgliederverwaltung, Eventplanung und Buchhaltung (ClubDesk-Contributors, 2025; WildApricot-Contributors, 2025). Motivierende oder interaktive Komponenten, wie sie in Gamification-Konzepten beschrieben werden, sind in diesen Systemen kein grosser Bestandteil. Diskussionen über eine

mögliche Erweiterung durch Belohnungsmechanismen existieren lediglich in Anwenderforen, was die bislang geringe praktische Umsetzung solcher Konzepte verdeutlicht ([WildApricot-Community, 2015](#)) .

Die vorliegende Arbeit positioniert sich zwischen diesen beiden Ansätzen, indem sie den organisatorischen Verwaltungsfokus klassischer Systeme mit der motivierenden Wirkung von Gamification-Elementen in [ClansCore](#) kombiniert. Durch die laufende Einführung des vorherigen Systems (Discord-Bot) beim Verein [EGSwiss](#) über das Teammitglied Vanessa Alves, während der Sommermonate 2025, konnten Rückmeldungen des Vorstandes gesammelt und praxisrelevante Anforderungen sowie Verbesserungsvorschläge identifiziert werden. Diese sind in der bestehenden Softwarelösung bislang unberücksichtigt geblieben sind. Damit leistet die Arbeit einen Beitrag zur Integration von Motivation, Datenmanagement und Vereinsorganisation in einem einheitlichen, plattformunabhängigen Anwendungskonzept ClansCore.

2 Anforderungen

Dieses Kapitel stellt die Anforderungen an ClansCore für den Verein systematisch dar. Die Anforderungen basieren auf klar definierten User Stories, welche die Bedürfnisse der Nutzer abbilden. Aus diesen User Stories werden die Functional Requirements abgeleitet, welche die konkreten Funktionen des Bots und des Dashboards spezifizieren. Darauf aufbauend definieren die Use Cases, wie diese Anforderungen in der geplanten Umsetzung realisiert werden. Neben den Functional Requirements werden auch die Non-Functional Requirements berücksichtigt, um Qualitätsmerkmale wie Sicherheit, Skalierbarkeit und Benutzerfreundlichkeit sicherzustellen. Zudem erfolgt eine Priorisierung der Anforderungen, damit die wichtigsten Funktionen frühzeitig implementiert werden. Im Gegensatz zum Vorprojekt liegt der Schwerpunkt auf der Erweiterung des bestehenden Discord-Bots zu einem modularen und sicheren Verwaltungssystem mit Web-Dashboard (siehe Abschnitt 1.3).

2.1 Priorisierung der Anforderungen

Damit der Entwicklungsprozess auf die wichtigsten Features fokussiert bleibt, erhalten gewisse Anforderungen eine höhere Priorität als andere. Dadurch kann zügig ein Prototyp entwickelt, getestet und auf Basis von Nutzerfeedback iterativ verbessert werden. Die nicht optionalen Use Cases und Non-Functional Requirements bilden das MVP (Minimum Viable Product) und stellen die funktionale Basis des Projekts dar.

Die nachfolgende Auflistung beschreibt die Prioritäts-Kategorien und deren Bedeutung:

Prioritäts-Kategorie	Beschreibung
Hoch (<u>MVP</u>) - Essenziell für die erste Version	Diese Anforderungen sind direkt mit den Kernanforderungen des MVP verknüpft und müssen für eine funktionale Erstversion vorhanden sein.
Mittel - Wichtige Funktionen nach dem MVP	Diese Anforderungen ergänzen das System sinnvoll, sind jedoch nicht kritisch für die erste Version.
Niedrig - Erweiterungen für spätere Versionen	Diese Anforderungen verbessern die Benutzerfreundlichkeit und Verwaltung, sind aber nicht erforderlich für den ersten MVP-Release.

Tab. 1: Beschreibung der Prioritäts-Kategorien

2.2 Legende zum Erfüllungsgrad der Anforderungen

Die Resultate der Anforderungen werden jeweils in den Anforderungstabellen dokumentiert. Zusätzlich wird der Status farblich hervorgehoben, um eine schnelle Übersicht über den Erfüllungsgrad zu ermöglichen.

Farbe	Status
 	Ziel der Anforderung wurde erreicht.
 	Ziel der Anforderung wurde zum Teil oder über einen anderen Weg erreicht.
 	Ziel der Anforderung wurde nicht erreicht.

Tab. 2: Beschreibung der Resultats-Kategorien

2.3 User Stories

Die nachfolgenden User Stories beschreiben die Erwartungen und Bedürfnisse der Nutzer an das aus dem Vorprojekt erweiterte System ClansCore. Sie werden aus der Sicht der Endanwender formuliert und bilden die Grundlage für die Ableitung der Functional Requirements. Die Analyse berücksichtigt die Rollen Vorstand, Mitglied, Nicht-Mitglied sowie die Systemperspektive Verein.

Als Verein...

- möchte ich ClansCore plattformunabhängig und modular betreiben, um nicht auf Discord beschränkt zu sein und weitere Plattformen einfach anbinden zu können.
- möchte ich eine zentrale Übersicht über Rollen, Events, Aufgaben und Aktivitäten haben, um operative und strategische Entscheide zu treffen.

- möchte ich Verlässlichkeit und Vertrauen durch Datenintegrität, Sicherheit und regelmässige Sicherungen / Überwachung sicherstellen.

Als Vorstandsmitglied...

- möchte ich Mitglieder, Rollen und Events im Admin-Dashboard verwalten, inklusive Rechte und Synchronisation mit Discord.
- möchte ich Aufgaben und Gamification erfassen und steuern sowie Analysen zu Aktivität und Engagement erhalten.
- möchte ich Zahlungserinnerungen und Zahlungsbestätigungen automatisiert erhalten, um administrativen Aufwand zu reduzieren.
- möchte ich Nachvollziehbarkeit durch Änderungs- und Transaktionsprotokolle sowie Datenintegritätsprüfungen.
- möchte ich sicheren Administrationszugriff sowie Monitoring und Alarme bei Konflikten und Sicherheitsereignissen.

Als Mitglied...

- möchte ich Aufgaben, Punkte und Ranglisten plattformunabhängig einsehen, jedoch konsistent zu Discord.
- möchte ich transparente Punkteberechnung verstehen und mich für Aufgaben anmelden.
- möchte ich meine Datenschutzrechte (Auskunft, Löschung, Einwilligung) selbst ausüben.

Als Nicht-Mitglied...

- möchte ich mich ausserhalb von Discord bewerben und den Status meiner Bewerbung einsehen.

2.4 Functional Requirements

Die Functional Requirements leiten sich aus den User Stories ab und spezifizieren die erwarteten Funktionen von ClansCore. Sie beschreiben, welche neuen oder erweiterten Features im Rahmen dieser Arbeit implementiert werden müssen, um die in der Einleitung formulierten Ziele zu erreichen (siehe Abschnitt 1.3). Im Gegensatz zum Vorprojekt liegt der Fokus auf Plattformunabhängigkeit, Sicherheit, Datenintegrität und Erweiterbarkeit durch ein webbasiertes Dashboard.

ID	FR1
Name	Login und Authentifizierung
Beschreibung	• Das Admin-Dashboard muss ein sicheres Loginsystem mit Benutzername und Passwort bereitstellen. • Das System muss optional eine <u>Multi-Faktor-Authentifizierung</u> (MFA) unterstützen. • Authentifizierungsvorgänge müssen protokolliert werden.

Tab. 3: Beschreibung Functional Requirement 1

ID	FR2
Name	Online-Mitgliedsbewerbung
Beschreibung	• Das Dashboard muss ein Bewerbungsformular für Nicht-Mitglieder bereitstellen. • Bewerbungen müssen automatisch in der Datenbank gespeichert und dem Vorstand im Dashboard angezeigt werden. • Bewerbende müssen den Status ihrer Bewerbung einsehen können.

Tab. 4: Beschreibung Functional Requirement 2

ID	FR3
Name	Passwortmanagement
Beschreibung	• Benutzer des Dashboards müssen ihr Passwort selbstständig ändern oder zurücksetzen können. • Passwörter müssen verschlüsselt gespeichert und sicher übermittelt werden.

Tab. 5: Beschreibung Functional Requirement 3

ID	FR4
Name	Verwaltung von Mitgliedern, Rollen und Events
Beschreibung	Das Admin-Dashboard muss Funktionen zur Verwaltung bereitstellen, die es erlauben: • Mitgliederprofile und deren Rollen anzuzeigen und zu bearbeiten • Events zu erstellen, zu bearbeiten und zu löschen • Erinnerungen für Events automatisch zu generieren • Änderungen müssen synchron mit der Discord-Datenbank erfolgen

Tab. 6: Beschreibung Functional Requirement 4

ID	FR5
Name	Steuerung und Rechteverwaltung
Beschreibung	• Das Dashboard muss erlauben, die Zugriffsrechte für Bot-Funktionen und Dashboard-Bereiche zu verwalten. • Es muss konfigurierbar sein, welche Rollen welche Aktionen ausführen dürfen. • Änderungen an Berechtigungen müssen automatisch protokolliert werden.

Tab. 7: Beschreibung Functional Requirement 5

ID	FR6
Name	Gamification-Parameterverwaltung
Beschreibung	• Das Dashboard muss die Erfassung, Anpassung und Gewichtung von Gamification-Parametern ermöglichen. • Bei der Erstellung von Aufgaben muss das System automatisch eine empfohlene Punktezahl vorschlagen.

Tab. 8: Beschreibung Functional Requirement 6

ID	FR7
Name	Gamification-Datenanalyse
Beschreibung	• Das Dashboard muss detaillierte Statistiken zu Aufgaben, Belohnungen und Aktivitäten bereitstellen. • Mitgliederaktivitäten müssen in Echtzeit ausgewertet und visualisiert werden können.

Tab. 9: Beschreibung Functional Requirement 7

ID	FR8
Name	Gamification-Ansicht für Mitglieder
Beschreibung	• Mitglieder müssen im Dashboard ihre Punkte, Aufgaben und Ranglistenstände einsehen können. • Alle angezeigten Werte müssen aus der gleichen Datenquelle wie Discord stammen, um Konsistenz sicherzustellen.

Tab. 10: Beschreibung Functional Requirement 8

ID	FR9
Name	Aufgabenverwaltung und Anmeldung
Beschreibung	• Das Dashboard muss die Erstellung und Verwaltung von Aufgaben ermöglichen. • Mitglieder müssen sich über das Dashboard für Aufgaben anmelden können. • Der Fortschritt und Abschlussstatus muss vom Vorstand überprüft und freigegeben werden können.

Tab. 11: Beschreibung Functional Requirement 9

ID	FR10
Name	Zahlungserinnerung
Beschreibung	• Das System muss periodisch prüfen, ob ein Mitglied seinen Mitgliederbeitrag bezahlt hat. • Bei ausstehenden Zahlungen muss automatisch eine Erinnerung über Discord oder das Dashboard versendet werden. • Die Erinnerung darf erst nach Ablauf des definierten Vereinsjahres erfolgen.

Tab. 12: Beschreibung Functional Requirement 10

ID	FR11
Name	Zahlungsbestätigung
Beschreibung	• Nach der Erfassung einer eingegangenen Zahlung muss das System dem Vorstand automatisch eine Bestätigung übermitteln. • Die Zahlungsinformationen müssen revisionssicher gespeichert werden.

Tab. 13: Beschreibung Functional Requirement 11

ID	FR12
Name	Plattformunabhängigkeit und Erweiterbarkeit
Beschreibung	• ClansCore muss so aufgebaut sein, dass neben Discord weitere bestehende Plattformen integriert werden können. • Gemeinsame Funktionen (z. B. Mitgliederverwaltung, Aufgaben, Rollen, Gamification) müssen über eine zentrale Logik- und Datenschicht verfügbar sein. • Die Kommunikation zwischen Plattformen und Kernsystem erfolgt über klar definierte Schnittstellen (z. B. REST, WebSocket oder Event-Queue). • Neue Plattformen sollen ohne tiefgreifende Änderungen am Kernsystem hinzugefügt werden können.

Tab. 14: Beschreibung Functional Requirement 12

2.5 Use Cases

Die Use Cases beschreiben detailliert die Interaktionen zwischen Nutzern und ClansCore. Sie zeigen, wie die Functional Requirements umgesetzt werden. Darüber hinaus konkretisieren sie Anforderungen an Sicherheit, Skalierbarkeit und Benutzerfreundlichkeit.

ID	UC1
Name	Login und Authentifizierung (<u>FR1</u>)
Akteur	Mitglied, Vorstand
Beschreibung	Der Zugriff auf das Admin-Dashboard erfordert eine Authentifizierung (optional wird <u>MFA</u> unterstützt).
Voraussetzung	Ein gültiger Dashboard-Account besteht.
Standard-Verlauf	1. Der Benutzer gibt Benutzernamen und Passwort ein. 2. ClansCore prüft die Zugangsdaten. 3. (Optional) Das System fordert den zweiten Faktor an und validiert ihn. 4. Bei Erfolg wird der Benutzer zur Startseite weitergeleitet.
Alternativ-Verlauf	• Ungültige Zugangsdaten führen zur Fehlermeldung. • Benutzer hat das Passwort vergessen und kann es zurücksetzen (<u>UC3</u>).
Nachbedingung	Der Benutzer ist eingeloggt und der Zugriff wird protokolliert.
Priorität	Hoch
Resultat	

Tab. 15: Beschreibung Fully dressed Use Case 1

ID	UC2
Name	Online-Mitgliedsbewerbung (<u>FR2</u>)
Akteur	Nicht-Mitglied, Vorstand
Beschreibung	Nicht-Mitglieder bewerben sich ausserhalb von Discord über das Dashboard. Der Prozess ist transparent und nachvollziehbar.
Voraussetzung	Dashboard ist mit der Vereinsdatenbank verbunden.
Standard-Verlauf	1. Nicht-Mitglied füllt das Bewerbungsformular aus und stimmt der Datenverarbeitung zu. 2. ClansCore speichert die Bewerbung und informiert den Vorstand im Dashboard. 3. Der Vorstand prüft die Angaben und entscheidet (Annahme / Ablehnung). 4. Bei Annahme wird der Mitgliedsstatus gesetzt und die Synchronisation mit Discord ausgelöst. 5. Bewerbende erhalten eine Status-Benachrichtigung.
Alternativ-Verlauf	• Unvollständige / fehlerhafte Angaben führen zu einem Validierungsfehler und Korrekturaufforderung. • Ablehnung führt zu einer Benachrichtigung mit Begründung (sofern vorgesehen).
Nachbedingung	Es wird im System dokumentiert und Rollen / Synchronisation sind konsistent.
Priorität	Hoch
Resultat	

Tab. 16: Beschreibung Fully dressed Use Case 2

ID	UC3
Name	Passwort zurücksetzen (<u>FR3</u>)
Akteur	Mitglied, Vorstand
Beschreibung	Benutzer setzen ein vergessenes Passwort sicher zurück.
Voraussetzung	Ein Dashboard-Account existiert.
Standard-Verlauf	1. Benutzer startet „Passwort vergessen“. 2. Das System stellt einen sicheren Reset-Prozess bereit. 3. Benutzer setzt ein neues Passwort und meldet sich an (<u>UC1</u>).
Alternativ-Verlauf	• Wenn kein Account vorhanden ist, führt das zum Abbruch mit Hinweis. • Wenn der Token abgelaufen / ungültig ist, wird ein neuer Reset-Prozess erforderlich.
Nachbedingung	Das neue Passwort ist gespeichert und das Ereignis protokolliert.
Priorität	Hoch
Resultat	

Tab. 17: Beschreibung Fully dressed Use Case 3

ID	UC4
Name	Passwort ändern (<u>FR3</u>)
Akteur	Mitglied, Vorstand
Beschreibung	Benutzer ändern ihr Passwort selbstständig.
Voraussetzung	Benutzer ist eingeloggt.
Standard-Verlauf	1. Benutzer gibt altes und neues Passwort ein. 2. Das System verifiziert das alte Passwort und speichert das neue.
Alternativ-Verlauf	• Falls das alte Passwort falsch ist, wird eine Fehlermeldung ausgegeben und die Änderung abgebrochen.
Nachbedingung	Das neue Passwort ist aktiv und die Änderung protokolliert.
Priorität	Mittel
Resultat	

Tab. 18: Beschreibung Fully dressed Use Case 4

ID	UC5
Name	Verwaltung von Mitgliedern, Rollen und Rechten (FR4 , FR5)
Akteur	Vorstand
Beschreibung	Der Vorstand verwaltet Mitglieder, Rollen, Events und Berechtigungen zentral im Dashboard.
Voraussetzung	Dashboard ist mit der Vereinsdatenbank verbunden und das Vorstandsmitglied ist eingeloggt.
Standard-Verlauf	1. Vorstand ruft den Verwaltungsbereich auf. 2. Anzeigen / Bearbeiten von Mitgliedern, Rollen und Events. 3. Konfiguration von Zugriffsrechten und erlaubten Aktionen. 4. Änderungen werden gespeichert und synchronisiert.
Alternativ-Verlauf	• Ein Validierungsfehler führt zur Fehlermeldung und die Änderung wird nicht übernommen.
Nachbedingung	Änderungen sind konsistent persistiert und protokolliert.
Priorität	Hoch
Resultat	

Tab. 19: Beschreibung Fully dressed Use Case 5

ID	UC6
Name	Gamification-Parameter verwalten (FR6)
Akteur	Vorstand
Beschreibung	Der Vorstand pflegt die Gewichtungen / Parameter, welche für Punktevorschläge verwendet werden.
Voraussetzung	Vorstandsmitglied ist eingeloggt.
Standard-Verlauf	1. Öffnen der Parameterverwaltung. 2. Anpassen der Parameter und Speichern.
Alternativ-Verlauf	Unvollständige / fehlerhafte Eingaben ergeben einen Validierungsfehler.
Nachbedingung	Parameter sind aktuell und künftige Punktevorschläge berücksichtigen die Anpassungen.
Priorität	Hoch
Resultat	

Tab. 20: Beschreibung Fully dressed Use Case 6

ID	UC7
Name	Aufgabenverwaltung mit Punktevorschlag (FR6 , FR9)
Akteur	Vorstand
Beschreibung	Der Vorstand erstellt und veröffentlicht Aufgaben. ClansCore schlägt basierend auf Parametern eine Punktezah vor.
Voraussetzung	Dashboard ist verbunden und das Vorstandsmitglied ist eingeloggt. Die Gamification-Parameter sind gepflegt.
Standard-Verlauf	1. Vorstand erfasst Aufgabe (Metadaten, Kriterien). 2. System zeigt einen berechneten Punktevorschlag. 3. Vorstand bestätigt / ändert und veröffentlicht die Aufgabe.
Alternativ-Verlauf	• Wenn keine Parameter vorhanden sind, gibt es keinen Vorschlag. Die Aufgabe kann dennoch mit manueller Punktezah angelegt werden. • Ein Validierungsfehler führt zur Fehlermeldung.
Nachbedingung	Aufgabe ist veröffentlicht und im System gespeichert.
Priorität	Hoch
Resultat	

Tab. 21: Beschreibung Fully dressed Use Case 7

ID	UC8
Name	Ansicht Aufgaben, Ranglisten und eigene Punkte (FR7 , FR8)
Akteur	Mitglied, Vorstand
Beschreibung	Mitglieder sehen Aufgaben, Ranglisten und ihren Punktestand. Der Vorstand erhält zusätzliche Analyseansichten.
Voraussetzung	Dashboard ist verbunden und der Benutzer ist eingeloggt.
Standard-Verlauf	1. Mitglied ruft die Gamification-Ansicht auf und sieht Aufgaben, Ranglisten sowie den eigenen Punktestand. 2. Vorstand ruft Analyse-Dashboard auf (Aggregationen / Trends).
Alternativ-Verlauf	• Wenn keine Aufgaben verfügbar sind, werden entsprechende Hinweise angezeigt.
Nachbedingung	Gamification-Informationen werden konsistent und aktuell angezeigt.
Priorität	Hoch
Resultat	

Tab. 22: Beschreibung Fully dressed Use Case 8

ID	UC9
Name	Anmeldung zu Aufgaben (FR9)
Akteur	Mitglied, Vorstand
Beschreibung	Mitglieder melden sich über das Dashboard für Aufgaben an. Die Freigabe erfolgt durch den Vorstand.
Voraussetzung	Mitglied ist eingeloggt und die Aufgabe veröffentlicht.
Standard-Verlauf	1. Mitglied wählt eine Aufgabe und meldet sich an. 2. System speichert die Anmeldung. 3. Vorstand prüft Abschluss und gibt Punkte frei.
Alternativ-Verlauf	• Ein Fehler bei der Anmeldung führt zu einer Fehlermeldung und erneute Eingabe.
Nachbedingung	Anmeldung / Abschlussstatus sind korrekt gespeichert und Punkte entsprechend gutgeschrieben.
Priorität	Hoch
Resultat	

Tab. 23: Beschreibung Fully dressed Use Case 9

ID	UC10
Name	Benachrichtigungen und Informationen zu Zahlungen (FR10 , FR11)
Akteur	Mitglied, Vorstand, System
Beschreibung	Beiträge werden jährlich fällig. ClansCore automatisiert die Erinnerung und die revisionssichere Rückmeldung zu eingegangenen Zahlungen.
Voraussetzung	Ein Mitglied hat einen ausstehenden Mitgliederbeitrag. Zahlungsdaten sind im System hinterlegt.
Standard-Verlauf	1. Das System prüft periodisch Fälligkeiten und identifiziert ausstehende Zahlungen. 2. Das System versendet eine Zahlungserinnerung (Discord oder Dashboard-Benachrichtigung). 3. Die Zahlung wird erfasst (z. B. manuelle Verbuchung oder Import). 4. Das System speichert Datum und Referenz revisionssicher. 5. Der Vorstand erhält automatisch eine Zahlungsbestätigung im Dashboard.
Alternativ-Verlauf	• Zahlung kann nicht eindeutig zugeordnet werden: der Vorstand wird zur Klärung aufgefordert (offene Position im Dashboard). • Bei erneuter Nichtzahlung nach Erinnerung erscheint der Fall in der Liste „Überfällig“. Der Vorstand entscheidet das weitere Vorgehen.
Nachbedingung	Der Zahlungsstatus des Mitglieds ist aktuell und protokolliert.
Priorität	Mittel
Resultat	

Tab. 24: Beschreibung Fully dressed Use Case 10

ID	UC11
Name	Weitere Plattform-Integration (FR12)
Akteur	Nicht-Mitglied, Mitglied, System
Beschreibung	ClansCore stellt prototypisch Funktionen ausserhalb von Discord oder Dashboard bereit (Bewerbung, Datenabfrage) und synchronisiert zentrale Daten.
Voraussetzung	Neue Plattform ist mit der zentralen Datenbank verbunden.
Standard-Verlauf	1. Nicht-Mitglied reicht Bewerbung über die Plattform ein (UC2). 2. System persistiert Daten und informiert den Vorstand. 3. Vorstand entscheidet. Bei Annahme wird Mitgliedsstatus gesetzt und Daten synchronisiert. 4. Mitglied kann auf der neuen Plattform eigene Personendaten einsehen.
Alternativ-Verlauf	• Ablehnung führt zu keiner Aufnahme und die bewerbende Person wird informiert.
Nachbedingung	Zentrale Daten sind konsistent und Plattformunabhängigkeit ist prototypisch nachgewiesen.
Priorität	Niedrig
Resultat	

Tab. 25: Beschreibung Fully dressed Use Case 11

2.6 Non-Functional Requirements

Die Non-Functional Requirements definieren die Qualitätsmerkmale von ClansCore. Sie betreffen Sicherheit, Wartbarkeit, Zuverlässigkeit, Kompatibilität und Benutzerfreundlichkeit. Jede Anforderung ist überprüfbar und wird im Projektverlauf gemessen oder getestet.

ID	NFR1
Beschreibung	Die Codequalität erfüllt die im Projekt definierten Linting- und Formatierungsregeln. In der CI-Pipeline dürfen keine Fehler vom Typ „error“ auftreten.
Anforderungen	Maintainability - Code Quality
Priorität	Hoch
Messung	Automatische Prüfung mit Linter in der CI-Pipeline. Ziel: 0 Fehler im Hauptbranch.
Testen	Bei jedem Commit.
Resultat	

Tab. 26: Beschreibung Non-Functional Requirement 1

ID	NFR2
Beschreibung	Die Testabdeckung der Kernkomponenten (Bot, Dashboard) beträgt mindestens 70%.
Anforderungen	Maintainability - Testability
Priorität	Hoch
Messung	Automatische Berechnung über das Test-Framework (Coverage Report).
Testen	Bei jedem Release und nach grösseren Codeänderungen.
Resultat	

Tab. 27: Beschreibung Non-Functional Requirement 2

ID	NFR3
Beschreibung	Das System darf keine unbehandelten Fehler verursachen, die zu einem Absturz oder Datenverlust führen. Fehlermeldungen müssen verständlich und benutzerfreundlich angezeigt werden.
Anforderungen	Reliability - Fault Tolerance
Priorität	Hoch
Messung	Manuelle Fehlertests. Erfolgsbedingung: Keine unbehandelten Exceptions in Logs, keine Systemabstürze.
Testen	Am Ende der Implementierungsphase.
Resultat	

Tab. 28: Beschreibung Non-Functional Requirement 3

ID	NFR4
Beschreibung	Das Admin-Dashboard verwendet ein einheitliches und benutzerfreundliches Design mit klarer Navigation und unmittelbarem Feedback.
Anforderungen	Usability - Consistency
Priorität	Mittel
Messung	Usability-Test mit mindestens 3 Testpersonen. Erfolgsziel: 80% der Aufgaben ohne Hilfe lösbar.
Testen	Am Ende der Implementierungsphase.
Resultat	

Tab. 29: Beschreibung Non-Functional Requirement 4

ID	NFR5	
Beschreibung	Das Dashboard ist mit den aktuellen Versionen von Google Chrome und Mozilla Firefox vollständig nutzbar.	
Anforderungen	Compatibility - Interoperability	
Priorität	Hoch	
Messung	Manueller Funktionstest auf beiden Browsern. Erfolgsziel: 100 % Funktionsumfang verfügbar.	
Testen	Am Ende der Implementierungsphase.	
Resultat		

Tab. 30: Beschreibung Non-Functional Requirement 5

ID	NFR6	
Beschreibung	Personenbezogene Daten werden vertraulich behandelt und nach dem Prinzip „Privacy by Design“ verarbeitet. Daten werden verschlüsselt gespeichert und übertragen.	
Anforderungen	Security - Confidentiality	
Priorität	Hoch	
Messung	Prüfung der Verschlüsselungskonfiguration (HTTPS, Datenbankverschlüsselung).	
Testen	Vor Projektabschluss.	
Resultat		

Tab. 31: Beschreibung Non-Functional Requirement 6

ID	NFR7	
Beschreibung	Das Gamification-System darf nicht manuell manipuliert werden können. Punkteänderungen müssen nachvollziehbar protokolliert sein.	
Anforderungen	Security - Integrity	
Priorität	Hoch	
Messung	Überprüfung von 5 Stichproben in der Datenbank. Jede Änderung muss eindeutig einem Benutzer oder Prozess zuordenbar sein.	
Testen	Am Ende der Implementierungsphase.	
Resultat		

Tab. 32: Beschreibung Non-Functional Requirement 7

ID	NFR8	
Beschreibung	Nach einem erfolgreichen Commit und bestandenem Testlauf wird automatisch ein neues Build-Image erstellt und ausgeliefert.	
Anforderungen	Maintainability - Deployment Automation	
Priorität	Mittel	
Messung	CI/CD-Pipeline-Auswertung: Build- und Deploy-Prozess müssen ohne Fehler abgeschlossen werden.	
Testen	Nach jedem Commit auf dem Main Branch.	
Resultat		

Tab. 33: Beschreibung Non-Functional Requirement 8

ID	NFR9	
Beschreibung	Alle Änderungen an der Datenbank müssen im Log nachvollziehbar sein (Benutzer, Zeit, Aktion).	
Anforderungen	Security - Accountability	
Priorität	Mittel	
Messung	Manuelle Kontrolle von 10 zufälligen Datenbankeinträgen. Jede Änderung muss einem Benutzer zugeordnet sein.	
Testen	Am Ende der Implementierungsphase.	
Resultat		

Tab. 34: Beschreibung Non-Functional Requirement 9

ID	NFR10	
Beschreibung	Das Dashboard reagiert auf Benutzerinteraktionen innerhalb von 2 Sekunden.	
Anforderungen	Performance - Response Time	
Priorität	Mittel	
Messung	Stoppuhrmessung bei den wichtigsten Aktionen (Login, Seitenwechsel).	
Testen	Am Ende der Implementierungsphase.	
Resultat		

Tab. 35: Beschreibung Non-Functional Requirement 10

3 Erweitertes Gamification-Konzept

Bereits im Vorprojekt wurden erste Erkenntnisse zu Motivation, Hindernissen und Gamification im Vereinskontext erhoben. Diese basierten auf eine Nutzergruppen-Analyse mit Nicht-Mitglieder, Vereins-Mitglieder und Vorstands-Mitglieder. Die Ergebnisse lieferten eine breite, quantitative Datengrundlage, die allgemeine Tendenzen sichtbar machten, beispielsweise welche Aktivitäten besonders motivierend wirken oder welche Risiken bei der Einführung von Gamification-Systemen bestehen (Alves & Schnider, 2025).

Für die vorliegende Arbeit war es jedoch notwendig, über diese Vorarbeit hinauszugehen. Während die Umfragen des Vorprojekts vor allem die Perspektive der Mitglieder und Erfahrungswerte von Vanessa Alves als Präsidentin von EGSwiss abbildeten, sollen nun die Erfahrungen und Einschätzungen anderer Vereinsvorstände, systematisch untersucht werden. Präsident:innen und Vorstände sind zentrale Entscheidungsträger in Vereinen und können eine strategische Sicht auf Motivation und Engagement bieten. Ihre Sichtweise ist entscheidend, um zu verstehen, welche strukturellen und organisatorischen Massnahmen tatsächlich umsetzbar sind und wie digitale Tools wie der Discord-Bot oder ein Admin-Dashboard sie dabei unterstützen können.

3.1 Interviews zur Motivation von Vereinsmitgliedern

Das Ziel der Interviews ist es, einen praxisnahen Abgleich zwischen theoretischen Konzepten und organisatorischer Realität zu schaffen. Während die Vorarbeit vor allem beschrieb, was Mitglieder motiviert, soll jetzt untersucht werden, wie Vorstände Motivation fördern und welche Unterstützung sie dabei benötigen.

Konkret sollen die Interviews:

- **Motivationsstrategien der Vereine erfassen:** Welche Massnahmen nutzen Vorstände aktuell, um Mitglieder langfristig an den Verein zu binden?
- **Herausforderungen aus Leitungsperspektive beleuchten:** Welche strukturellen Hürden sehen Vorstände bei Engagement und Motivation?
- **Einsatz digitaler Werkzeuge untersuchen:** Welche Rolle spielen Plattformen wie Discord oder Social Media für Vorstände selbst?
- **Potenziale von Gamification im Management identifizieren:** Welche Chancen sehen Vorstände in Belohnungssystemen, und welche Risiken gilt es zu vermeiden?
- **Anforderungen an digitale Unterstützung ableiten:** Welche Features im Discord-Bot und welche Funktionen im Admin-Dashboard würden Vorstände konkret entlasten oder Motivation steigern?

Damit erweitern die Interviews den Erkenntnisstand aus der Vorarbeit um externe, strategische Führungsperspektiven.

3.1.1 Methodisches Vorgehen

Die Interviews wurden als halb-strukturierte, qualitative Gespräche konzipiert. Im Unterschied zur standardisierten Umfrage im Vorprojekt ermöglicht diese Methode detaillierte Einblicke und erlaubt es, individuell auf die Antworten einzugehen.

Die Auswahl der Interviewpartner erfolgte gezielt. Befragt wurden Präsident:innen und Vorstands-Mitglieder von Gaming-Vereinen, da sie einerseits die operative Situation im Verein kennen und andererseits Entscheidungen über zukünftige Massnahmen treffen. Diese Zielgruppe liefert somit nicht nur Einschätzungen zur Motivation der Mitglieder, sondern auch Anforderungen an Management- und Organisationswerkzeuge.

Die Interviews dauerten 30 bis 45 Minuten und folgten einem thematisch gegliederten Leitfaden. Die Gesprächspartner wurden dabei schrittweise von allgemeinen Fragen über Motivation und Aktivitäten hin zu digitalen Tools und schliesslich zu hypothetischen Feature-Ideen geführt.

3.1.2 Interviewleitfaden

Der Leitfaden wurde in sieben Blöcke gegliedert, die einen roten Faden gewährleisten:

1. **Allgemeine Fragen:** Kontext zu Rolle, Vereinsgrösse, Gründungsjahr, um eine Grundlage zur Vergleichbarkeit der Antworten zu schaffen.
2. **Vereinsmotivation:** aktuelle Beitrittsgründe, Bindungsfaktoren, Herausforderungen im Verein.
3. **Aktivitäten und Engagement:** motivierende Angebote, Unterschiede zwischen neuen und erfahrenen Mitgliedern, Umgang mit Passivität.
4. **Digitale Tools und Gamification:** Nutzung digitaler Plattformen, Erfahrungen mit spielerischen Elementen, Chancen und Risiken.
5. **Zukunft und Anpassungen:** Wünsche nach Belohnungen, nachhaltige Fördermassnahmen.
6. **Digitale Brücke:** Hypothetische Fragen zu einem digitalen Tool, Kennzahlen und gewünschte Features.
7. **Abschluss:** Platz für offene Ergänzungen.

Wichtig ist, dass die Interviewten erst in den letzten Blöcken mit der Möglichkeit digitaler Lösungen konfrontiert wurden. Dadurch konnten zunächst unverfälschte Aussagen zur Vereinsmotivation gewonnen werden, bevor ein Bezug zu einem Discord-Bot oder Admin-Dashboard hergestellt wurde.

3.1.3 Mehrwert gegenüber dem Vorprojekt

Die Interviews schaffen einen klaren Mehrwert gegenüber der Vorarbeit. Während die Umfragen des Vorprojekts Mitgliederperspektiven in quantitativer Form erfassten, liefern die Interviews qualitative Einblicke aus Führungssicht. Anstelle allgemeiner Trends stehen hier strategische und organisatorische Aspekte im Vordergrund, zum Beispiel wie Vorstände Motivation gezielt steuern oder wo sie digitale Unterstützung benötigen. Die Ergebnisse sind nicht nur eine Wiederholung, sondern dienen dazu, konkrete Anforderungen für die Weiterentwicklung des Discord-Bots und Erstellung des Admin-Dashboards abzuleiten. Die Ergebnisse liefern einen wertvollen Einblick in die Fragen nach Kennzahlen und Management-Funktionen im Dashboard.

Damit stellen die Interviews ein zentrales Bindeglied zwischen Theorie (Motivations- und Gamification-Modelle), Praxis (Mitgliederumfragen aus der Vorarbeit) und der konkreten Softwareentwicklung in dieser Bachelorarbeit dar.

3.2 Octalysis Framework

Das folgende Kapitel basiert inhaltlich auf dem Werk Actionable Gamification: Beyond Points, Badges and Leaderboards von Yu-kai Chou (2015). Sämtliche Konzepte, Modelle und Begriffe, sofern nicht anders gekennzeichnet, stammen aus diesem Werk. ([Chou, 2015](#))

Das Octalysis Framework ist aus der Gamification Forschung und stellt acht Core Drives vor, die zentrale Motivationsfaktoren menschlichen Verhaltens darstellen. Die Anwendung des Octalysis-Modells erfolgt durch die Analyse, welche Elemente eines Produkts oder Systems auf welche der acht Core Drives wirken. Diese Mechaniken werden in das Modell übertragen und gewichtet, um ein Gesamtbild darüber zu erhalten, welche motivationalen Anreize im Vordergrund stehen.

Die acht Core Drives von dem Octalysis Modell sind:

1. Epic Meaning & Calling

Das Gefühl ein Teil von etwas grösserem zu sein. In Videospielen wird dieses Gefühl oft am Anfang angesprochen in dem klar gemacht wird, dass nur der Spieler die Welt retten kann.

2. Development & Accomplishment

Die Motivation von Fortschritt, neue Fähigkeiten erlernen und Herausforderungen überwinden. Bei dieser Art von Motivation helfen oft Punktesysteme, Trophäen und Ranglisten.

3. Empowerment of Creativity & Feedback

Diese Motivationsart trifft zu wenn Benutzer ihre Kreativität ausleben können, eine Rückmeldung bekommen und dann wieder Anpassungen machen können. Ein gutes Beispiel hierfür sind Legos.

4. Ownership & Possession

Beschreibt die Motivation, die aus dem Gefühl des Besitzes entsteht. Dies kann sich beispielsweise auf gesammelte Punkte in einem System oder auf ein individuell gestaltetes Profil beziehen.

5. Social Influence & Relatedness

Dieser Core Drive vereint zentrale soziale Motivationsfaktoren wie soziale Anerkennung, Zugehörigkeitsgefühl, Wettbewerb und auch Neid. Wenn ein Freund etwas Besonderes besitzt oder nutzt kann das oft dazu motivieren das selbe anzustreben.

6. Scarcity & Impatience

Dieser Core Drive beinhaltet etwas zu wollen weil es selten, nur für kurze Zeit oder für den Moment gar nicht erhältlich ist.

7. Unpredictability & Curiosity

Dieser Core Drive basiert auf der psychologischen Reaktion auf unerwartete Ereignisse. Wenn Abläufe nicht vorhersehbar sind, wird die kognitive Aufmerksamkeit erhöht. Dies entsteht sowohl bei Glücksspielmechanismen als auch bei alltäglichen Unterhaltungsformaten wie Filmen oder Büchern.

8. Loss & Avoidance

Die Motivation, negative Konsequenzen zu vermeiden. Ein Beispiel dafür ist das Festhalten an etwas, weil bereits viel Zeit oder Aufwand investiert wurde. Auch zeitlich begrenzte Angebote oder auslaufende Chancen machen sich dies zunutze.

3.2.1 ClansCore im Octalysis Framework

Durch Anwendung des Octalysis-Modells wurde folgendes Modell erstellt.

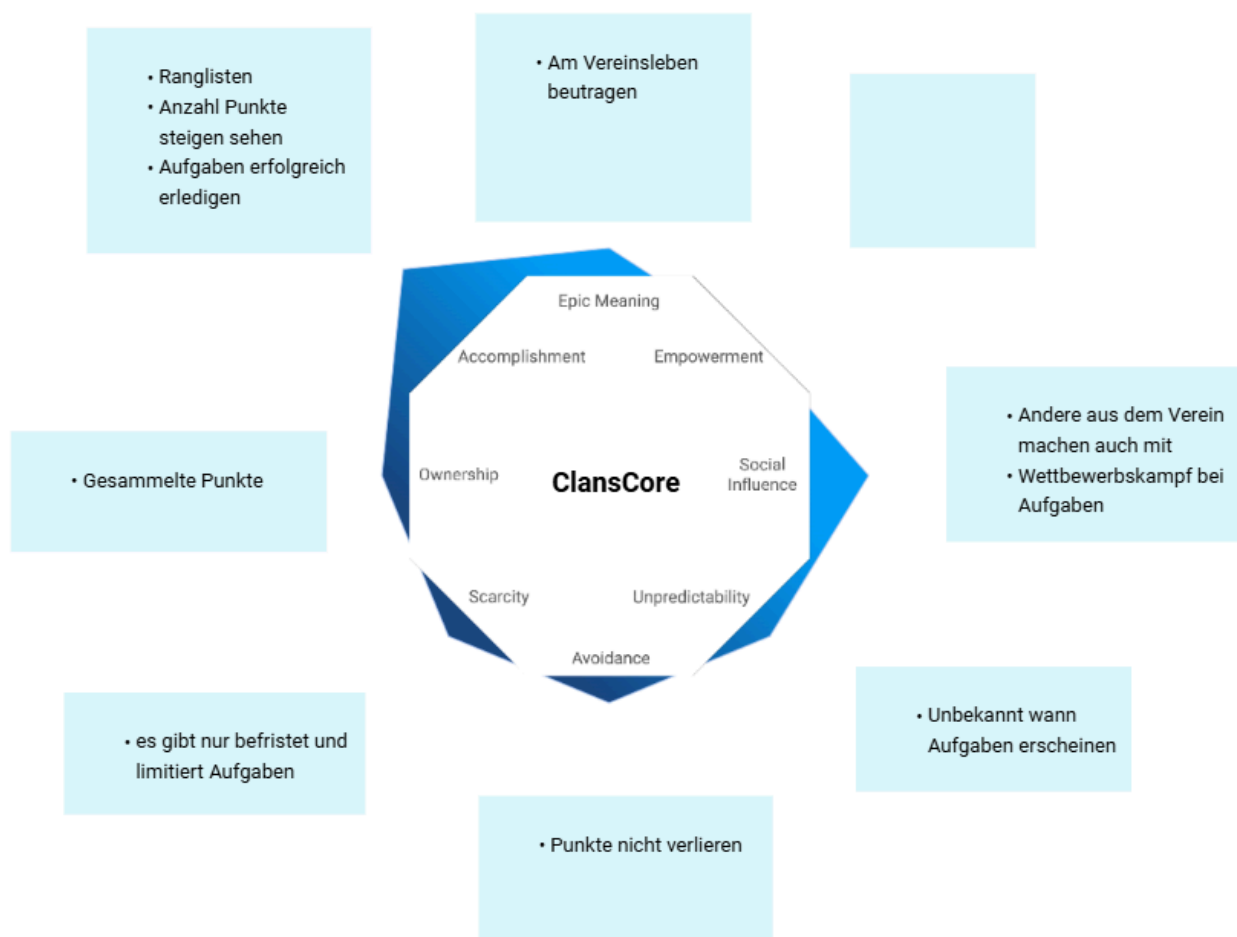


Abb. 1: ClansCore im Octalysis Modell

Die Anwendung des Octalysis-Frameworks auf ClansCore verdeutlicht, dass die Meisten Core Drives nur wenig angesprochen werden. Nur der Core Drive Development & Accomplishment ist stark ausgeprägt. Dieser Antrieb befindet sich auf der linken Seite des Modells und zählt zu den primär extrinsisch motivierenden Faktoren, da er auf Belohnungen wie Punkte, Status und sichtbaren Fortschritt abzielt. Im Gegensatz dazu bleibt der Core Drive Empowerment of Creativity & Feedback, der auf der rechten Seite des Octalysis-Modells angesiedelt ist und zu den intrinsischen Motivationsfaktoren zählt, in der aktuellen

Umsetzung unberücksichtigt. Insgesamt werden die intrinsischen motivierenden Faktoren zu wenig angesprochen. Eine stärkere Berücksichtigung intrinsischer Anreize würde nicht nur die Nutzerbindung erhöhen, sondern auch die Qualität der Beteiligung verbessern, da Handlungen weniger durch äusseren Druck, sondern durch persönliche Überzeugung getragen wären. Die intrinsischen Faktoren könnten durch geeignete Massnahmen, wie etwa durch Profilgestaltung, personalisierte Aufgaben oder besseren sozialen Funktionen, wie etwa ein Freundschaftssystem oder kooperative Aufgabenformate, verbessert werden. Eine solche Erweiterung würde die Plattform nicht nur motivierender gestalten, sondern auch die langfristige Nutzerbindung und die Identifikation mit der Vereinsplattform stärken.

4 Erweitertes Sicherheitskonzept

Das erweiterte Sicherheitskonzept definiert die konzeptionellen und organisatorischen Massnahmen, mit denen die Vertraulichkeit, Integrität und Verfügbarkeit der in ClansCore verarbeiteten Daten gewährleistet werden. Es baut auf den im Vorprojekt etablierten Grundlagen zu Datensicherheit, Rechteverwaltung und Datenschutz (vgl. Vorprojekt, Kap. 3 Sicherheitskonzept) auf, erweitert diese jedoch in Hinblick auf die neue Systemarchitektur mit einem webbasierten Dashboard und der synchronisierten Datenhaltung zwischen Discord und der Webplattform.

Das Backend, welches bereits im Vorprojekt als eigenständige, modulare Anwendung konzipiert wurde, bildet weiterhin das zentrale Bindeglied zwischen allen Plattformen. Neu hinzugekommen sind die Anforderungen an eine sichere Kommunikation zwischen Backend und Angular-Frontend sowie an eine konsistente und konfliktfreie Synchronisation von Benutzerdaten, Rollen und Gamification-Informationen über mehrere Zugriffspunkte. Die Systemarchitektur ist somit nicht nur funktional, sondern auch sicherheitstechnisch stärker verteilt, was eine Erweiterung des bisherigen Sicherheitsmodells erforderlich macht.

4.1 Ziel und Geltungsbereich

Ziel des erweiterten Sicherheitskonzepts ist es, die personenbezogenen und vereinsinternen Daten vor unbefugtem Zugriff, Manipulation und Verlust zu schützen. Der Fokus liegt auf Vertraulichkeit, Integrität und Verfügbarkeit, welche die drei zentralen Schutzziele der Informationssicherheit bilden (NIST, 2020).

Das Konzept erstreckt sich auf alle sicherheitsrelevanten Komponenten des Systems:

- **Backend** (Node.js / Express): zentrale Applikationslogik und API-Kommunikation
- **Discord-Bot** (Discord.js): Interaktionsschnittstelle für Mitgliederaktionen
- **Admin-Dashboard** (Angular): browserbasierte Verwaltungs- und Interaktionsplattform
- **Datenbank** (MongoDB): persistente Speicherung von Benutzer-, Rollen- und Aktivitätsdaten
- **Infrastruktur** (Docker, GitHub Actions): Containerisierung, CI/CD und Deployment
- **Externe Dienste** (Google Calendar API): Synchronisation von Kalendereinträgen

Nicht berücksichtigt werden die nativen Sicherheitsmechanismen externer Plattformen (Discord, Google), da sie ausserhalb der direkten Kontrolle des Systems liegen.

4.2 Sicherheitsprinzipien

Das Sicherheitskonzept orientiert sich an vier grundlegende Prinzipien der Informationssicherheit, welche als Fundament weiterer technischer und organisatorischer Sicherheitsmassnahmen dienen sowie in allen Systemschichten konsistent angewendet werden.

Prinzip	Beschreibung der Umsetzung
Least Privilege	• Alle Komponenten und Benutzerrollen besitzen nur jene Rechte, die für ihre Funktion notwendig sind. • Vorstandsmitglieder dürfen Daten auf Organisations-Ebene (Mitgliederverwaltung, Events, Punktesystem) bearbeiten. • Mitglieder dürfen ausschliesslich ihre eigenen Daten sehen und bearbeiten. • Öffentliche Informationen wie Vereins-Events, Bewerbungsformular, Statuten und Datenschutzrichtlinie sind ohne Login zugänglich.
Defense in Depth	Sicherheit wird durch mehrere, voneinander unabhängige Schutzebenen gewährleistet, etwa durch Transportverschlüsselung, Container-Isolation, Zugriffsbeschränkungen und rollenbasierte Rechteprüfung. (<u>Murugiah & Karen, 2017; NIST, 2020</u>)
Accountability	Alle sicherheitsrelevanten Aktionen (Login-Versuche, Rollenänderungen, Punkteänderungen) werden protokolliert und sind für berechtigte Personen nachvollziehbar.
Privacy by Design	Es werden nur notwendige Daten gespeichert, analog zum im Vorprojekt definierten Datensparsamkeitsprinzip (vgl. <u>Vorprojekt</u> , Kap. 3.3 ff.). Die Datenschutzinformationen sind zusätzlich im Dashboard direkt einsehbar.

Tab. 36: Beschreibung der vier grundlegenden Prinzipien

4.3 Authentifizierung und Autorisierung

Die Authentifizierung erfolgt über mehrere, voneinander getrennte Ebenen:

- **Discord-OAuth2** für Nutzerinteraktionen innerhalb der Plattform (unverändert aus dem [Vorprojekt](#), vgl. Kap. 3.2.1)
- **Dashboard-Authentifizierung** für den Webzugriff
- **Infrastruktur-Authentifizierung** für administrative Server- und Datenbankzugriffe

Das Backend fungiert dabei als zentrale Autorisierungsinstanz und verwaltet sämtliche Rollen- und Rechteinformationen. Es prüft eingehende Anfragen unabhängig von der Plattform, um konsistente Zugriffsbeschränkungen sicherzustellen.

Die Authentifizierung unterscheidet zwischen menschlichen Benutzer und automatisierten Dienstkonten. Menschliche Zugriffe erfolgen über eine interaktive Anmeldung und werden bei sensiblen Operationen durch eine [Multi-Faktor-Authentifizierung](#) (nachfolgend MFA genannt) ergänzt. Damit wird insbesondere der Zugriff auf Infrastruktur und Datenbanken gegen unautorisierte Übernahmen geschützt.

4.3.1 Dienstkonten und automatisierte Zugriffe

Im Gegensatz zu menschlichen Benutzer greifen automatisierte Systemkomponenten (z. B. der Discord-Bot, das Backend oder der CI/CD-Prozess) über sogenannte Dienstkonten auf Systemressourcen zu. Diese Konten sind speziell für maschinelle Abläufe vorgesehen, arbeiten mit sicheren Tokens oder Secrets und unterliegen restriktiven Rechten nach dem Prinzip Least Privilege.

Bereits im [Vorprojekt](#) war der Discord-Bot als solches Dienstkonto umgesetzt. Er lief in einem eigenen Container, authenticizierte sich über ein Bot-Token aus Umgebungsvariablen (.env) und kommunizierte über die Discord-API mit dem System. Da der Bot umfängliche Berechtigungen benötigt, wird er weiterhin Administratorrechte im Discord-Server erhalten. Diese Ausnahme wird bewusst akzeptiert, da sie für die technische Funktionsfähigkeit des Systems erforderlich ist.

Die Trennung zwischen menschlichen Admins (mit MFA-geschütztem Zugriff) und automatisierten Systemprozessen bleibt dennoch gewahrt. Dienstkonten werden systemweit durch [Role-Based Access Control](#) (nachfolgend RBAC genannt) begrenzt und besitzen nur die für ihren Anwendungsfall notwendigen Berechtigungen. Dadurch wird das Risiko einer Kompromittierung oder eines Missbrauchs erheblich reduziert.

4.3.2 Absicherung des Datenbankzugriffs

Die Datenbank (nachfolgend DB genannt) stellt das sicherheitskritischste Element des Systems dar, da sie sämtliche personenbezogenen und transaktionsbezogenen Daten des Vereins enthält. Entsprechend muss der Zugriff sowohl technisch als auch organisatorisch auf mehreren Ebenen geschützt werden. Die Absicherung umfasst einerseits systeminterne Verbindungen zwischen Backend, Bot und DB, andererseits den administrativen Zugang für menschliche Benutzer mit erweiterten Berechtigungen (Administratoren).

Im Rahmen der Konzeptentwicklung wurden drei Ansätze zur MFA des DB-Zugriffs untersucht. Hintergrund ist, dass MongoDB selbst keinen nativen Zwei-Faktor-Login unterstützt. Der zweite Faktor muss daher vorgelagert, also auf Netzwerkebene oder durch Besitzfaktoren wie Zertifikate, umgesetzt werden.

Variante	Beschreibung	Stärken (+) und Schwächen (-)
Netzwerk-MFA (VPN / SSH-Bastion)	Zugriff auf die Datenbank ist nur über ein privates, nicht öffentlich erreichbares Netzwerk möglich. Der Zugang zu diesem Netzwerk wird über eine Bastion oder einen VPN-Gateway geregelt, der eine Multi-Faktor-Authentifizierung (z. B. TOTP oder Hardware-Token) verlangt. Damit wird der gesamte Zugriffspfad geschützt, ohne dass Backend oder Bot angepasst werden müssen.	+ geringer Implementierungsaufwand, hohe Kompatibilität und transparente Integration in bestehende Infrastruktur. - VPN oder Bastion werden zu zentralen Kontrollpunkten, die gezieltes Monitoring und Backup-Strategien erfordern.
X.509-Clientzertifikate (mutual TLS)	Die Authentifizierung erfolgt über ein digitales Zertifikat als Besitzfaktor (MongoDB-Contributors, 2025a). Diese Methode bietet einen hohen Schutz vor Phishing und erlaubt eine präzise Nachverfolgbarkeit einzelner Zugriffe.	+ stark gegen Phishing und gute Auditierbarkeit. - aufwändige Zertifikatsverwaltung, zusätzlicher Administrationsaufwand.
SSO (Single-Sign-On) / Identity-Provider- basierte MFA	Authentifizierung erfolgt über einen externen Identity Provider (z. B. Microsoft Entra ID ¹) mittels OpenID Connect (nachfolgend OIDC genannt). MongoDB unterstützt seit Version 7.0 die direkte Integration solcher Anbieter, wodurch sich SSO und MFA zentral verwalten lassen (MongoDB-Contributors, 2025b).	+ zentrale Benutzer- und Richtlinienverwaltung über Systeme hinweg. - erfordert zusätzliche Infrastruktur, Lizenzkosten und laufende Wartung, welche für Non-Profit-Vereine intransparent und unverhältnismässig aufwendig ist.

Tab. 37: Varianten zur Absicherung des DB-Zugangs

4.3.2.1 Bewertung und Entscheid

Die Analyse zeigt, dass Netzwerk-MFA den besten Kompromiss zwischen Sicherheit, Wartbarkeit und Implementierbarkeit bietet. Sie schützt den gesamten Zugriffspfad zur Datenbank und kann ohne strukturelle Änderungen an bestehenden Komponenten eingeführt werden. In der bestehenden OST-Serverumgebung (virtuelle Linux-Server mit SSH-Zugang) lässt sich diese Methode schnell, kostengünstig und mit geringem Risiko implementieren. Darüber hinaus ist sie vollständig kompatibel mit der bestehenden Container- und CI/CD-Umgebung (Docker, GitHub Actions).

Die Alternative über X.509-Zertifikate als besonders sicher eingestuft, ist aber aufgrund des erhöhten Verwaltungsaufwands und der nötigen Zertifikatsinfrastruktur für den Anwendungsfall eines Vereins-Tools nur bedingt sinnvoll.

Eine SSO- / IdP-basierte Lösung bietet theoretisch die beste Richtliniensteuerung und zentrale Benutzerverwaltung, würde aber zusätzliche Infrastruktur, Lizenzen und Wartungskosten verursachen. Da ClansCore bewusst als Open-Source-System für Vereine konzipiert ist, stehen Kosteneffizienz und einfache Wartbarkeit im Vordergrund. Die Einführung eines kommerziellen Identity-Providers würde diesen Grundsatz unterlaufen und die Einstiegshürde für kleinere Vereine erheblich erhöhen.

Aus diesen Gründen wird für ClansCore die Lösung Netzwerk-MFA über VPN beziehungsweise SSH-Bastion gewählt. Diese Variante bietet eine robuste Absicherung gegen unautorisierte Zugriffe, ist wirtschaftlich umsetzbar und unterstützt das Prinzip der klaren Trennung zwischen menschlichen und maschinellen Zugängen. Menschliche Admins authentifizieren sich über MFA auf der Netzwerkebene und systeminterne Dienste (z. B. Bot, Backend) kommunizieren ausschliesslich innerhalb des privaten Docker-Netzwerks über dedizierte Dienstkonten mit minimalen Rechten.

Die Wirksamkeit dieser Architektur wird durch externe Quellen untermauert. Laut Microsoft verhindern aktivierte MFA-Mechanismen ca. 99.2% bis 99.9% der üblichen Kontoübernahmen und werden deshalb in Zukunft potenziell zwingend eingeführt für VPN / Bastion ([Microsoft-Contributors, 2025a; 2025b](#)). Ausserdem empfiehlt die Cybersecurity and Infrastructure Security Agency (CISA) den MFA-Einsatz, insbesondere für VPN- und Bastion-Zugänge ([CISA, 2022](#)). Damit erfüllt die gewählte Lösung die anerkannten Best Practices moderner IT-Sicherheitsarchitekturen und bleibt zugleich ressourcenschonend, was den spezifischen Anforderungen einer Vereinssoftware gerecht wird.

Zusätzlich nutzt die Datenbank das standardisierte Authentifizierungsverfahren SCRAM-SHA-256 ([MongoDB-Contributors, 2025c](#)) in Kombination mit rollenbasierter Zugriffskontrolle ([MongoDB-Contributors, 2025d](#)). Für zukünftige Ausbaustufen kann eine ergänzende Zertifikatsauthentifizierung für besonders privilegierte Konten in Betracht gezogen werden.

¹<https://www.microsoft.com/de-ch/security/business/identity-access/microsoft-entra-id>

4.3.2.2 Übertragbarkeit auf andere Vereine

Das vorgestellte Sicherheitskonzept berücksichtigt, dass ClansCore auch von Vereinen betrieben wird, die ihre Infrastruktur eigenständig hosten. Viele kleinere Organisationen verfügen über keinen dedizierten IT-Betrieb und müssen Sicherheit daher mit möglichst geringen Kosten und administrativem Aufwand gewährleisten. Für solche Vereine stellt das Modell der Netzwerk-MFA eine besonders praktikable Lösung dar. Es erfordert weder teure Identity-Management-Lösungen noch komplexe Zertifikatsverwaltungs-Systeme und kann mit frei verfügbaren Open-Source-Werkzeugen (z. B. OpenVPN²) umgesetzt werden. Dadurch bleibt das Sicherheitsniveau hoch, ohne dass der Betrieb zusätzliche finanzielle oder organisatorische Hürden verursacht. Diese Skalierbarkeit ist zentral, um den Einsatz von ClansCore auch in kleineren, ehrenamtlich geführten Vereinen langfristig sicherzustellen.

4.3.3 Dashboard-Login

Das Admin-Dashboard verfügt über eine eigenständige Authentifizierungsebene, die eine sichere Verwaltung von Vereinsdaten unabhängig von Discord ermöglicht. Die Benutzerverwaltung erfolgt im Backend, das sowohl die Authentifizierung als auch die Autorisierung des Angular-Frontends übernimmt.

Nach erfolgreicher Anmeldung erhalten Nutzer:innen eine kurzlebige Sitzungsberechtigung, die an ihre Rolle (Admin, Vorstand, Mitglied, Gast) gebunden ist. Rollenänderungen oder Sicherheitsvorfälle führen zum sofortigen Widerruf aktiver Sitzungen. Die Berechtigungsprüfung erfolgt ausschliesslich serverseitig im Backend, während Angular-Route-Guards nur der Benutzerführung dienen.

Das Backend (Node.js/Express) fungiert als zentrale Autorisierungsinstanz. Nach erfolgreicher Anmeldung werden kurzlebige Zugriffstokens und rotierende Auffrischungstokens ausgestellt, die als HttpOnly- und Secure-Cookies gespeichert werden. Zustandsverändernde Anfragen erfordern zusätzlich ein CSRF-Token, wodurch Manipulationsversuche (CSRF, XSS) wirksam verhindert werden. Diese Massnahmen entsprechen den Empfehlungen des Open Web Application Security Project (OWASP) zur sicheren Sitzungsverwaltung und CSRF-Prävention, insbesondere im Hinblick auf kurze Session-Lebenszyklen, sichere Cookie-Flags und den Einsatz des Double-Submit-Cookie-Patterns ([OWASP-Cheat-Sheets-Series-Contributors, 2025a; 2025b](#)).

Die gesamte Kommunikation zwischen Angular-Frontend und Backend erfolgt über TLS-verschlüsselte HTTPS-Verbindungen. Rollen und Berechtigungen werden über RBAC zentral im Backend definiert und bei jeder API-Anfrage geprüft. So können etwa Vorstandsmitglieder Vereinsdaten verwalten, während Mitglieder nur auf ihre eigenen Informationen zugreifen dürfen.

Für privilegierte Konten (Admin, Vorstand) ist eine optionale Mehrfaktorprüfung (MFA) vorgesehen, die bei Anmeldung einen zusätzlichen Besitzfaktor (TOTP) verlangt. Sämtliche sicherheitsrelevanten Ereignisse (z. B. Anmeldeversuche, Token-Erneuerungen oder Rollenänderungen) werden im Audit-Log protokolliert und sind für den Vorstand nachvollziehbar.

Diese Umsetzung kombiniert Benutzerfreundlichkeit mit hoher Sicherheit und folgt den Prinzipien Least Privilege, Defense in Depth und Accountability, wodurch die Integrität und Nachvollziehbarkeit aller administrativen Zugriffe gewährleistet ist.

4.4 Daten- und Schnittstellensicherheit

Die Kommunikation zwischen den Systemkomponenten erfolgt ausschliesslich über gesicherte Kanäle:

Verbindung	Technologie	Absicherung
Dashboard ↔ Backend	REST-API / HTTPS	TLS 1.3 + JWT-Auth
Backend ↔ Discord-Bot	WebSocket Event-Bridge	Bot-Token + Signaturprüfung
Backend ↔ MongoDB	TCP / TLS	SCRAM-SHA-256 + RBAC + Netzwerk-MFA
Backend ↔ Google API	OAuth2 Client-Secrets	Zugriffstoken mit Scopes

Tab. 38: Beschreibung der abgesicherten Kanäle in ClansCore

4.4.1 Eingabevalidierung

Alle externen Eingaben (REST, WebSocket, Formular) werden serverseitig validiert und sanitisiert (Injection-Schutz). Fehler werden kontrolliert behandelt und geloggt, ohne sensitive Details preiszugeben.

²<https://openvpn.net/>

4.4.2 Synchronisierung & Konfliktlösung

Da ClansCore sowohl über den Discord-Bot als auch über das Admin-Dashboard oder anderen potentiellen Plattformen genutzt wird, können Änderungen an denselben Datensätzen parallel erfolgen. Beispielsweise kann ein Vorstandsmitglied ein Mitgliederprofil im Dashboard anpassen, während der Bot gleichzeitig eine Punktebuchung durchführt. Um daraus entstehende Datenkonflikte zu vermeiden, wird im Backend eine optimistische Nebenläufigkeitskontrolle (Optimistic Concurrency Control, OCC) angewendet.

Jeder Datensatz in der Datenbank enthält eine Versionskennung sowie einen Zeitstempel. Bei einem Update-Vorgang überprüft das Backend, ob die vom Client übermittelte Version noch mit der aktuellen Version in der Datenbank übereinstimmt. Wurde der Datensatz in der Zwischenzeit verändert, wird der Schreibvorgang abgewiesen und als Konflikt im Audit-Log vermerkt. Das Dashboard fordert daraufhin den aktuellen Datensatz an und kann die Änderungen manuell oder automatisch zusammenführen.

Für besonders sensible Vorgänge, wie Punktebuchungen, Rollenänderungen oder Event-Updates, werden zusätzlich atomare Transaktionen eingesetzt. Dadurch ist gewährleistet, dass zusammenhängende Operationen entweder vollständig oder gar nicht ausgeführt werden, was Inkonsistenzen in der Datenbank verhindert.

Alle Konflikte sowie deren Auflösung werden im Audit-Log dokumentiert. Dieses Verfahren stellt sicher, dass Datenintegrität und Nachvollziehbarkeit auch bei paralleler Nutzung durch mehrere Plattformen gewährleistet bleiben. Es basiert auf anerkannten Architekturmustern zur Transaktions- und Konsistenzsicherung in verteilten Systemen ([Martin, 2003](#)).

4.5 Datenschutz und DSGVO/DSG-Umsetzung

Die im Vorprojekt formulierten Datenschutzrichtlinien (vgl. [Vorprojekt, Kap. 3.3 - 3.5](#)) gelten unverändert.

Neu können Nutzer direkt im Dashboard ihre gespeicherten Informationen einsehen, die Einwilligung zur Datenspeicherung widerrufen oder eine Datenlöschung anfordern. Das Hosting erfolgt auf Servern in der Schweiz ohne US-Rechtsbezug, wodurch die Datenhoheit gewahrt bleibt und der Zugriff durch ausländische Behörden (z. B. über den CLOUD Act) ausgeschlossen ist ([U.S.-Department-of-Justice, 2019](#)). Die Verarbeitung personenbezogener Daten richtet sich nach den Grundsätzen der DSGVO ([Europäische-Union, 2016](#)), welche Privacy by Design and by Default voraussetzen, sowie dem schweizerischen Datenschutzgesetz ([Schweizerische-Eidgenossenschaft, 2020](#)).

4.6 Server- und Deployment-Sicherheit

Das Backend wird in einer containerisierten Umgebung betrieben, die nach dem Prinzip Defense in Depth aufgebaut ist. Ziel ist eine mehrschichtige Absicherung, bei der einzelne Schutzebenen unabhängig voneinander wirken. ([NIST, 2020](#))

Die Architektur besteht aus separaten Containern für Discord-Bot, Backend-Service, Admin-Dashboard, Reverse-Proxy und MongoDB. Jeder Container besitzt ein eigenes Dateisystem, eigene Prozessräume und dedizierte Netzwerk-Namespaces. Die Kommunikation erfolgt ausschliesslich über interne Docker-Netzwerke. Externe Zugriffe laufen ausschliesslich über den Reverse-Proxy mit TLS 1.3-Verschlüsselung. Die Datenbank ist nicht öffentlich erreichbar (siehe [Abschnitt 4.3.2](#)).

Container-Isolation basiert auf Linux-Namespaces, Cgroups und Capability-Drops, die laut NIST als Kernmechanismen moderner Container-Sicherheit gelten sowie die Angriffsfläche erheblich reduzieren. ([Murugiah & Karen, 2017](#))

Zugriff auf den Host erfolgt ausschliesslich über SSH-Key-Authentifizierung mit MFA über VPN / Bastion-Ebene ([CISA, 2022](#)). Das Deployment erfolgt automatisiert über GitHub Actions, wobei sämtliche Secrets (z. B. Tokens, Zugangsdaten) in verschlüsselten GitHub-Vaults verwaltet werden. Backups sind versioniert, verschlüsselt und nur für Admins zugänglich.

Durch die Kombination von Container-Isolation, Netzwerksegmentierung und Multi-Faktor-Authentifizierung entsteht eine robuste, mehrschichtige Sicherheitsarchitektur, welche die Vertraulichkeit, Integrität und grundlegende Verfügbarkeit des Systems sicherstellt. Eine umfassende Gewährleistung der Verfügbarkeit würde jedoch zusätzliche Massnahmen erfordern, wie beispielsweise eine Systemverteilung (Distribution), Lastverteilung (Load Balancing) oder automatisierte Failover-Mechanismen. Diese Aspekte gehen über den Rahmen der vorliegenden Arbeit hinaus und werden daher nicht weiter berücksichtigt.

4.7 Manipulations- und Integritätsschutz

Wie in der Vorarbeit beschrieben (vgl. [Vorprojekt, Kap. 3.3](#)) werden alle Punkte- und Event-Transaktionen manipulations-sicher in der Datenbank protokolliert. Neu ergänzt sind:

- **Transaktionale Speicherung** mittels atomarer Operationen wie MongoDB Sessions ([MongoDB-Contributors, 2025e](#)) .
- **Audit-Log** aller Änderungen, insbesondere von Gamification-Daten, sichtbar für den Vorstand.
- Optionaler **Hash-Check** für Integritätsprüfung einzelner Datensätze.

Damit bleibt jede Änderung rückverfolgbar und das Vertrauen in die Datenbasis langfristig erhalten.

4.8 Risikoanalyse und Massnahmen

Die zentrale Sicherheitsrisiken umfassen unautorisierte Zugriffe, Datenlecks, Social-Engineering-Angriffe und Datenverlust. Für jedes Risiko existieren die folgenden präventiven und reaktive Massnahmen.

Risiko	Auswirkungsgrad	Massnahme	Status
Datenleck durch API	Hoch	HTTPS, Token-Auth, Input-Validation	Offen
Unautorisierter DB-Zugriff	Hoch	Netzwerk-MFA + SCRAM-SHA-256 + RBAC	Offen
Social Engineering	Mittel	Schulung	Offen
Datenverlust	Hoch	Regelmässige Backups + Restore-Tests	Offen

Tab. 39: Einschätzung der Risiken und deren Massnahmen

4.9 Monitoring und Wartung

Ein konsistentes Logging-Konzept dokumentiert sicherheitsrelevante Ereignisse (Auth-Flows, Rollenänderungen, fehlgeschlagene Zugriffe, Konfliktauflösungen). Für schwerwiegende Vorkommnisse sind Benachrichtigungen an einen Discord-Admin-Kanal und das Admin-Dashboard vorgesehen. Wiederkehrende Sicherheitsüberprüfungen orientieren sich an der OWASP-Top-10-Checkliste ([OWASP-Contributors, 2021](#)) . Konfigurations- und Rechteprüfungen erfolgen regelmässig.

Diese Massnahmen schaffen einen kontinuierlichen Überblick über die Systemintegrität und ermöglichen ein rasches Eingreifen bei Sicherheitsvorfällen.

5 Systemanalyse, Wireframes und Architektur

5.1 Wireframes

Zur Skizzierung des Aufbaus und der Struktur der ClansCore-Webseite wurde das Wireframe-Tool Balsamiq verwendet. Es wurden nicht für alle Seiten Wireframes erstellt, da diese in erster Linie dazu dienen, einen allgemeinen Überblick über die Gestaltung und Funktionalität von ClansCore zu vermitteln.

Nicht eingeloggt

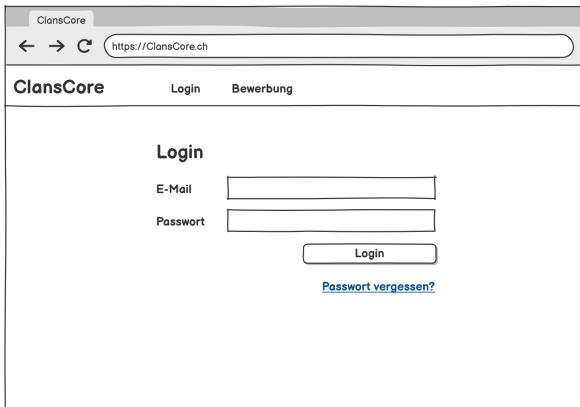


Abb. 2: Login

Login

Um die Kernfunktionen von ClansCore nutzen zu können, ist eine Anmeldung erforderlich. Auf der Login-Seite erfolgt die Authentifizierung durch die Eingabe der E-Mail-Adresse sowie des zugehörigen Passworts. Falls das Passwort vergessen wurde, besteht über einen entsprechenden Link die Möglichkeit, ein neues Passwort anzufordern.

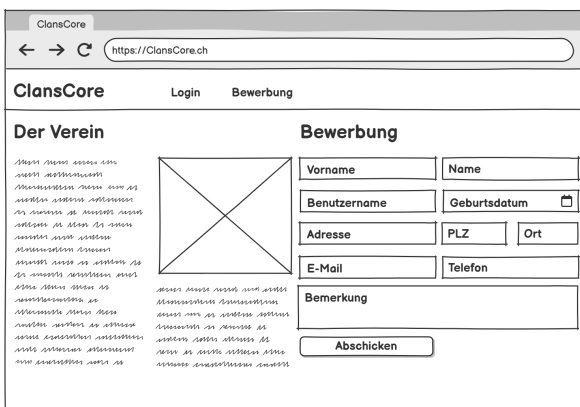


Abb. 3: Bewerbungsformular

Bewerbung

Auf der Bewerbung-Seite findet man auf der linken Seite Informationen über den Verein. Auf der rechten Seite befindet sich das Bewerbungsformular um dem Verein beizutreten.

Mitglied

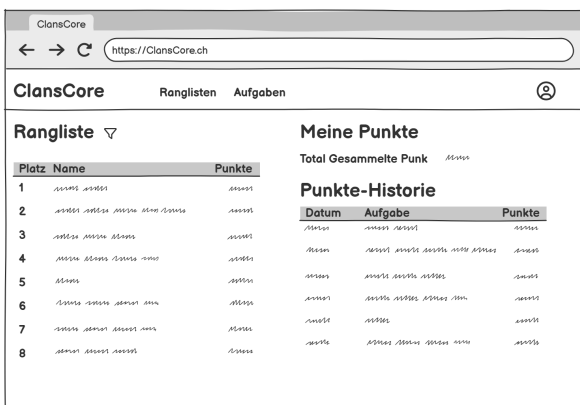


Abb. 4: Ranglisten-Seite

Ranglisten

Auf der Ranglisten-Seite befindet sich auf der linken Seite eine filterbare Rangliste, die einen Überblick über die Platzierungen der Nutzer bietet. Auf der rechten Seite werden die eigenen Punkte sowie deren Historie angezeigt.

Abb. 5: Aufgaben-Seite

Abb. 6: Mitglieder-Seite

Auf der Aufgaben-Seite werden alle verfügbaren Aufgaben angezeigt. Diese lassen sich nach verschiedenen Kriterien filtern. Solange eine Aufgabe noch nicht abgeschlossen ist, besteht die Möglichkeit, sich dafür anzumelden. Vorstandsmitglieder haben zusätzlich die Berechtigung, Aufgaben zu bearbeiten und hinzuzufügen.

Zugriff auf die Mitglieder-Seite haben ausschliesslich Vorstandsmitglieder. Es können bestehende Mitglieder gefiltert, bearbeitet, hinzugefügt oder gelöscht werden.

6 Qualitätssicherung

7 Implementation

8 Fazit

Kapitel II

Projekt Dokumentation

9 Projekt Plan

9.1 Zusammenarbeit

Als Projektmanagement-Methode wird Kanban verwendet welches es erlaubt das Projekt agil durchzuführen. Aufgrund der Teamgrösse von zwei Personen wurde sich gegen Scrum entschieden. Der zusätzliche organisatorische Mehraufwand durch Daily Meetings, Sprint Reviews und weitere Scrum Meetings würde keinen grossen Nutzen bringen. Dank der geringen Teamgrösse ist es kein Problem den Überblick über das Projekt zu behalten, sodass eine schlanke Methode wie Kanban eine passende Wahl ist.

Die Zusammenarbeit und Aufgabenaufteilung wird mittels Jira organisiert.

9.2 Meetings

Jede Woche findet ein Meeting unter den Teammitgliedern und ein Meeting mit dem Referenten statt.

Im Team-Meeting wird besprochen, welche Aufgaben fertiggestellt wurden und diese gegebenenfalls zusammen angeschaut. Zudem werden die Aufgaben für kommende Woche verteilt.

Beim Meeting mit dem Referenten wird der aktuelle Stand besprochen. Ausserdem kann dieses genutzt werden, um allfällige Fragen zu stellen.

9.3 Minimum Viable Product (MVP)

Das Minimum Viable Product (MVP) beinhaltet die Kernfunktionen des Projektes. Diese sind folgendermassen definiert:

- **Erweiterung Discord-Bot:** Ziel ist es, den bestehenden Discord-Bot basierend auf dem während des Einführungsprozesses für den Verein EGSwiss erhaltenen Feedback gezielt weiterzuentwickeln. Dabei sollen Funktionen neu erstellt, ergänzt und verbessert werden.
- **Erweiterung Gamification:** Ziel ist es, das bestehende Gamification-System auszubauen und zu untersuchen, welche Erweiterungen die Motivation und das Engagement im Verein steigern können.
- **Plattform-Unabhängigkeit:** Ein Admin-Dashboard soll erstellt werden, welches dem Vorstand Anpassungen an bereits erfassten Daten ermöglicht, ohne direkten Zugriff auf die Datenbank. Das Admin-Dashboard soll unabhängig von Discord nutzbar sein und somit die Applikation plattformunabhängig machen.

9.4 Long-Term Plan

Der Long-Term Plan wurde in 5 verschiedenen Phasen aufgeteilt. Diese werden in der folgenden Tabelle genauer beschrieben. Optionale Aufgaben stehen in Klammern.

Phase	Woche	Datum	Ziele	Meilenstein
Inception	1	15.09 - 21.09.	Einrichtung, MVP definieren	
	2	22.09. - 28.09.	Projektplan, Risikomanagement, Long-Term-Plan	M1
Elaboration	3	29.09. - 05.10.	Requirements, Einleitung, Stand der Technik	
	4	06.10. - 12.10.	Interviews andere Vereine, Sicherheitskonzept	
	5	13.10. - 19.10.	Gamifikationkonzept, Architektur, Mockups GUI	
	6	20.10. - 26.10.	Testkonzept, Prototyp Admin-Dashboard (Proof of Concept)	M2
Construction	7	27.10. - 02.11.	Refactoring, 1.1, 1.2, 7.1	
	8	03.11. - 09.11.	2.1, 2.2, 2.3, 3.1, 3.2, 3.3	
	9	10.11. - 16.11.	4.1, 4.2, 6.1, 6.2, 7.2	M3
	10	17.11. - 23.11.	4.3, 4.4, (5.1), (5.2), (6.3), 7.3	
	11	24.11. - 30.11.	(7.4), 8.1, 8.2, (8.3)	
	12	01.12. - 07.12.	Feedback aus Usability Test umsetzen und Implementation beenden	M4
Transition	13	08.12. - 14.12.	Dokumentation, Abschluss	
	14	15.12. - 19.12.	Fertigstellung, Abgabe Abstract und Dokumentation	M5
Presentation	15	20.12. - 28.12.	Präsentation und A0-Poster	
	16	29.12.2025 - 04.01.2026		
	17	05.01. - 09.01.		M6

Tab. 40: Long-Term Plan

Die einzelnen Komponenten und Funktionen sind wie folgt definiert:

1. Backend

- 1.1: Feedback / Neue Features ergänzen
- 1.2: Auto. Zahlungsbestätigung über Bank

2. Datenbank

- 2.1: Schema anpassen
- 2.2: DB-Login verbessern (2FA)
- 2.3: DB-Transactions / Rollbacks

3. Discord-Bot

- 3.1: Feedback / Neue Features ergänzen
- 3.2: Darstellung Events verbessern
- 3.3: Logs einsehen

4. Admin-Dashboard

- 4.1: Login erstellen
- 4.2: Darstellung / Bearbeitung DB-Daten (Mitglieder, Rollen, Events, Logs)
- 4.3: Erstellung des Punktesystems (Punkte, Belohnungen, Rangliste)
- 4.4: Darstellung / Bearbeitung für Mitglieder

5. Neue Plattform (optional)

- 5.1: Wahl der Plattform
- 5.2: Implementierung gleicher Features wie Discord-Bot

6. Unit-Tests

- 6.1: Discord-Bot
- 6.2: Dashboard
- 6.3: Neue Plattform

7. Pipelines CI/CD

- 7.1: Dokumentation
- 7.2: Discord-Bot
- 7.3: Dashboard
- 7.4: Neue Plattform

8. Usability Tests

- 8.1: Discord-Bot
- 8.2: Dashboard
- 8.3: Neue Plattform

9.4.1 Inception (Initiierung)

In der Inception-Phase werden die Anforderungen für das Projekt und der Projektplan erstellt, sowie die Risiken identifiziert. In dieser Phase wird zudem die generelle Herangehensweise und die Machbarkeit des Projektes bestimmt.

9.4.2 Elaboration (Ausarbeitung)

In der Elaboration-Phase werden die Anforderungen des Projektes im Detail analysiert. Hier wird das Gamification- und Sicherheitskonzept ausgearbeitet. Die Architektur wird mittels des C4-Modells definiert und ein erster Softwareprototyp wird erstellt. Dieser wird verwendet, um erste Systeme zu testen, auf denen dann später aufgebaut wird.

9.4.3 Construction (Konstruktion)

Die Construction-Phase wird dazu verwendet das Projekt technisch umzusetzen. In dieser Phase werden die geplanten Features umgesetzt und systematisch getestet. Usability-Tests werden durchgeführt auf dessen Basis Anpassungen vorgenommen werden. Die umgesetzten Features und Tests werden dokumentiert.

9.4.4 Transition (Übergang)

In der Transition-Phase wird das Projekt abgeschlossen. Die Dokumentation wird fertiggestellt und der Abstract wird geschrieben. Die Dokumentation wird in dieser Phase abgegeben.

9.4.5 Presentation (Präsentation)

Da das Projekt neben einer Studienarbeit auch eine Bachelorarbeit ist, wird in der Presentation-Phase die Präsentation erstellt. In dieser Phase arbeitet Vanessa Alves, welche die Bachelorarbeit durchführt, an der Präsentation und am Poster alleine.

9.4.6 Meilensteine

Die folgenden Meilensteine dienen als Hilfe, um die definierten Ziele des Projektes zu erreichen. Sie gelten als erfüllt, sobald die einzelnen Komponenten durch das Team in [Jira](#) als erledigt gekennzeichnet und vom Referenten eingesehen wurden.

M1: Projekt Initiierung

- Der Projektplan ist definiert.
- Die Risiken sind definiert.
- Das MVP ist definiert.
- Der Long-Term-Plan ist definiert.

M2: Prototyp

- Die Requirements sind definiert.
- Die Architektur ist definiert.
- Mockups für das Admin-Dashboard sind erstellt.
- Das Gamificationkonzept, Datenschutzkonzept und Testkonzept ist erstellt.
- Der erste Prototyp für das Admin-Dashboard ist erstellt.

M3: Erreichung des MVP

- Die Discord-Bot Erweiterungen sind umgesetzt.
- Das Admin-Dashboard ist umgesetzt.
- Das MVP wird erreicht.

M4: Fertigstellung der Implementation

- Alle geplanten, nicht-optionalen Features sind umgesetzt.
- Alle Funktionen sind erfolgreich getestet.
- Usability-Tests sind durchgeführt und Feedback ist umgesetzt.
- Die Testabdeckung entspricht dem definiertem Wert.

M5: Abgabe

- Die umfassende Dokumentation ist fertiggestellt.
- Das Abstract ist fertiggestellt.

M6: Präsentation

- Die Präsentation ist fertiggestellt.
- Das Poster ist fertiggestellt.

9.5 Risikomanagement

Die folgenden Tabellen zeigen die Risiken des Projektes auf und die Strategien die verwendet werden, um diese zu vermindern.

Wahrscheinlichkeit / Schweregrad	1 - Sehr Unwahrscheinlich	2 - Unwahrscheinlich	3 - Gelegentlich	4 - Wahrscheinlich	5 - Oft
4 - Katastrophal	R2				
3 - Kritisch		R3, R7	R9, R11		
2 - Signifikant			R4, R8, R13	R1, R12	
1 - Gering		R5		R6, R14	

Tab. 41: Risiko Matrix

ID	R1
Risiko	Lernkurve
Kommentar	Der Einsatz neuer Technologien (z. B. Frameworks, CI/CD, Web-Technologien) führt zu einer erhöhten Einarbeitungszeit.
Prävention	Frühzeitige Prototypen reduzieren die Einarbeitungszeit. Wissen wird im Team dokumentiert und geteilt.
Korrektur	Komplexe Features werden in kleinere Arbeitspakete unterteilt. Bei Problemen erfolgt gegenseitige Unterstützung im Team.

Tab. 42: Beschreibung Risiko 1

ID	R2
Risiko	Probleme bei Discord oder der neuen Plattformen
Kommentar	Discord oder alternative Plattformen (z. B. MS Teams, Matrix) können temporär nicht verfügbar sein.
Prävention	Evaluierung alternativer Plattformen und Absicherung kritischer Funktionen über die Web-Applikation.
Korrektur	Risiko muss weitgehend akzeptiert werden. Bei längeren Ausfällen wird die Nutzung der Web-Applikation verstärkt.

Tab. 43: Beschreibung Risiko 2

ID	R3
Risiko	Ressourcen limitiert
Kommentar	Ein Teammitglied fällt temporär oder dauerhaft aus.
Prävention	Alle Arbeitsschritte und Entscheidungen werden systematisch dokumentiert, um Wissenstransfer sicherzustellen.
Korrektur	Aufgaben werden durch verbleibende Teammitglieder übernommen. Fokus liegt auf priorisierten Features, um Projektziele trotz reduzierter Kapazität zu erreichen.

Tab. 44: Beschreibung Risiko 3

ID	R4
Risiko	Scope Creep
Kommentar	Anforderungen werden unklar definiert oder nachträglich erweitert, was den Projektumfang vergrössert.
Prävention	Detaillierte Anforderungsdefinition und Scope-Management im Rahmen der Meetings.
Korrektur	Nachträglich eingebrachte oder optionale Anforderungen mit niedriger Priorität werden in spätere Phasen verschoben oder gestrichen.

Tab. 45: Beschreibung Risiko 4

ID	R5
Risiko	Kompatibilitätsprobleme
Kommentar	Unterschiedliche Versionen von Tools, Bibliotheken oder Frameworks können zu Integrationsproblemen führen.
Prävention	Einheitliche Dokumentation der eingesetzten Versionen und Verwendung von Lockfiles (z. B. package-lock.json).
Korrektur	Versionskonflikte werden durch Abstimmung im Team gelöst, eine einheitliche Version wird verbindlich definiert.

Tab. 46: Beschreibung Risiko 5

ID	R6
Risiko	Zeiteinschätzung
Kommentar	Arbeitsaufwände werden zu optimistisch oder pessimistisch eingeschätzt.
Prävention	Iterative Abschätzung in den wöchentlichen Meetings sowie Nutzung von Erfahrungswerten und Vergleich mit vorherigen Projekten.
Korrektur	Optional geplante Features werden verschoben oder gestrichen, um Kernziele nicht zu gefährden.

Tab. 47: Beschreibung Risiko 6

ID	R7
Risiko	Änderungen in der Discord API oder andere Technologien
Kommentar	Fundamentale Änderungen in der Discord API oder anderen zentralen Technologien (z. B. Datenbank, Framework).
Prävention	Keine Verwendung von Funktionen, die als „deprecated“ markiert sind und Monitoring von Release Notes.
Korrektur	Anpassung des Codes und Refactoring bei API-Änderungen, auch wenn zusätzlicher Aufwand entsteht.

Tab. 48: Beschreibung Risiko 7

ID	R8
Risiko	Der bestehende Code ist stark an Discord gekoppelt, was die Entwicklung einer Web-App erschwert.
Kommentar	Der bisherige Code ist nicht komplett von Discord entkoppelt.
Prävention	Frühzeitiges Refactoring und Einführung einer modularen Architektur.
Korrektur	Falls Aufwand höher als geplant, Anpassung des Zeitplans und Reduktion optionaler Features.

Tab. 49: Beschreibung Risiko 8

ID	R9
Risiko	Dashboard ist nicht benutzerfreundlich
Kommentar	Dashboard erfüllen nicht die Erwartungen der Mitglieder/Vorstände hinsichtlich Usability.
Prävention	Entwicklung mit Usability-Tests und Einbindung von Stakeholdern.
Korrektur	Anpassung der Benutzeroberfläche und Interaktionskonzepte anhand von Feedback.

Tab. 50: Beschreibung Risiko 9

ID	R10
Risiko	Manipulation Gamification-System
Kommentar	Mitglieder könnten das Punktesystem missbrauchen (z. B. durch Mehrfachaktionen).
Prävention	Definition klarer Regeln, technische Prüfungen gegen Mehrfacheinträge, Monitoring verdächtiger Aktivitäten durch den Vorstand.
Korrektur	Rücksetzen oder Anpassung von Punkten bei Missbrauch und Anpassung der Regeln.

Tab. 51: Beschreibung Risiko 10

ID	R11
Risiko	Datenschutz
Kommentar	Potenzielle Risiken im Umgang mit personenbezogenen Daten.
Prävention	Erstellung eines Datenschutzkonzepts in Anlehnung an DSGVO und Schweizer Datenschutzgesetz sowie die Minimierung der erhobenen Daten.
Korrektur	Einholung von Einverständniserklärungen.

Tab. 52: Beschreibung Risiko 11

ID	R12
Risiko	Stakeholder-Anforderungen nicht berücksichtigt
Kommentar	Feedback von Mitgliedern oder Vorstand wird nicht ausreichend eingebunden.
Prävention	Priorisierung der Anforderungen, Usability-Tests mit realen Mitgliedern.
Korrektur	Anpassungen anhand von Feedback, Verschiebung von Features niedriger Priorität.

Tab. 53: Beschreibung Risiko 12

ID	R13
Risiko	Plattform-Integration komplexer als erwartet
Kommentar	Die geplante Entkopplung auf Dashboard oder anderen Plattformen verursacht höheren Aufwand oder technische Probleme.
Prävention	Proof-of-Concept frühzeitig erstellen, modulare Architektur.
Korrektur	Fokus auf Admin-Dashboard, weitere Plattformintegration in spätere Phase verschieben.

Tab. 54: Beschreibung Risiko 13

ID	R14
Risiko	Hosting oder CI/CD Ausfall
Kommentar	Server oder CI/CD-Pipeline fallen aus und blockieren Entwicklung oder Betrieb.
Prävention	Monitoring und automatisierte Backups, klare Verantwortlichkeiten im Team.
Korrektur	Deployment auf alternative Cloud-Umgebung oder manuelles Deployment als Notfalllösung.

Tab. 55: Beschreibung Risiko 14

9.5.1 Ausweichplan

Der Ausweichplan dient dazu systematisch auf Risiken zu reagieren, die im Vorfeld nicht bestimmt werden konnten. Der Ablauf ist wie folgt definiert:

1. Das Problem innerhalb einer Aufgabe wird identifiziert und mit dem Teammitglied besprochen.
2. Es wird versucht das Problem so schnell wie möglich zu lösen.
3. Kann das Problem nicht gelöst werden, wird es im nächsten Meeting mit dem Referenten besprochen.

Ziel dieses Ausweichplans ist es, bei auftretenden Problemen, die den Projektverlauf beeinträchtigen, eine klare und strukturierte Vorgehensweise zu gewährleisten. Dadurch kann effizient reagiert und ein reibungsloser Projektfortschritt sichergestellt werden.

10 Arbeitszeitnachweis

11 Tooling und Technologie-Entscheidungen

Dieses Kapitel beschreibt die im Rahmen der Bachelorarbeit eingesetzten Werkzeuge, Technologien und Entwicklungsprozesse. Es begründet die wesentlichen Entscheidungen hinsichtlich Projektorganisation, Dokumentation, Code-Qualität und Deployment. Grundlage bilden die Erfahrungen aus dem Vorprojekt, welche in dieser Arbeit systematisch weiterentwickelt werden.

11.1 Jira

Die Zusammenarbeit und Aufgabenverteilung wird mit Jira³ verwaltet. Für jede Aufgabe wird im Kanban Board ein Issue erstellt mit den folgenden Merkmalen:

- **Epic:** Die Phase zu dem das Issue gehört.
- **Typ:** Der Typ des Issues (Meeting, Aufgabe).
- **Geschätzte Zeit:** Die Soll-Zeit des Issues.
- **Verwendete Zeit:** Die tatsächlich aufgewendete Zeit um, den Issue zu beenden.
- **Zugewiesene Person:** Die Person, die das Issue durchführt.

Die Issues werden alle zuerst im Backlog definiert. Im wöchentlichen Team-Meeting werden die Issues, welche in dieser Woche geplant sind in die „Todo“-Spalte geschoben. Wenn die Person, der das Issue zugeteilt wurde, mit der Arbeit beginnt, wird dieser in die „In Progress“-Spalte geschoben. Wurde das Issue umgesetzt, kommt er in die „Review“-Spalte. Alle Issues in der Review Spalte werden im wöchentlichen Meeting besprochen und bei einer erfolgreichen Umsetzung in die „Done“-Spalte verschoben. Falls es noch Verbesserungspotenzial gibt, kommt das Issue zurück in die „Todo“-Spalte.

11.2 Teams

Für die interne Kommunikation, spontane Absprachen und die Planung von ausserordentlichen Meetings wird Microsoft Teams⁴ verwendet. Es dient als zentraler Kommunikationskanal und Speicherort für geteilte Dokumente. Damit wird ein kontinuierlicher Informationsfluss sichergestellt, was insbesondere bei verteilter Arbeit im Rahmen der Bachelorarbeit entscheidend ist.

11.3 GitHub

Das Vorprojekt wurde zu Beginn der Arbeit von GitLab⁵ auf GitHub⁶ migriert, um die Entwicklungs- und Veröffentlichungsprozesse langfristig unabhängig von der OST-internen Infrastruktur zu gestalten. Die zuvor auf der OST-GitLab gehostete Umgebung war nur innerhalb der Hochschule zugänglich, wodurch externe Zusammenarbeit und eine spätere Öffnung des Projekts erschwert wurden. GitHub bietet im Gegensatz dazu eine öffentlich zugängliche, weit verbreitete Plattform, die sich in der Open-Source-Community etabliert hat. Durch die Nutzung von GitHub Organisations und integrierten CI/CD-Workflows (GitHub Actions) kann die Projektstruktur klar gegliedert und zukünftige Erweiterungen effizient verwaltet werden. Neben diesen Vorteilen diente der Wechsel ausserdem dem Erkenntnisgewinn für das Team.

Für die Projektstruktur wurde eine eigene Organisation namens „ClansCore“⁷ erstellt. Innerhalb dieser Organisation befinden sich die nachfolgenden Repositories:

Repository	Beschreibung
clanscore ⁸	Der Quellcode der Applikation.
clanscore-dok ⁹	Die Projek-Dokumentation.
clanscore-dok-pub ¹⁰	Die automatisch generierten PDF-Versionen der Dokumentation und Sitzungs-Protokolle, welche über GitHub Pages veröffentlicht werden (siehe <u>Abschnitt 11.4.2</u>).

³<https://www.atlassian.com/software/jira>

⁴<https://www.microsoft.com/de-ch/microsoft-teams/log-in?market=ch>

⁵<https://gitlab.ost.ch/>

⁶<https://github.com/>

⁷<https://github.com/ClansCore>

⁸<https://github.com/ClansCore/clanscore>

⁹<https://github.com/ClansCore/clanscore-doc>

Tab. 56: Übersicht der Repositories innerhalb der GitHub-Organisation ClansCore

Während der Bearbeitung dieser Arbeit sind die Haupt-Repositories privat gestellt, um den Entwicklungsprozess geschützt zu halten. Die Organisation selbst ist jedoch öffentlich, wodurch Transparenz, Auffindbarkeit und spätere Zusammenarbeit gewährleistet bleiben.

11.3.1 GitHub Guidelines

1. Auf dem Dokumentations-Repository hat jedes Teammitglied seinen eigenen Branch im Format „dev-Initialen“ (z.B. dev-jt)
2. Auf den Code-Repositories (Discord-Bot, Admin-Dashboard) soll für jedes Feature oder Fehler ein jeweiliger Branch im Format „feature-FeatureName“ oder „bug-BugName“ erstellt werden.
3. Das Deployment der Repositories wird je auf einem eigenen Branch namens „deploy“ umgesetzt. Die Sammlung der Commits werden beim mergen des Main-Banches mithilfe der Funktion „Squash“ zusammengefasst, um die Anzahl ähnlicher Commit-Nachrichten zu reduzieren.
4. Bevor Änderungen beim Code in den Develop-Branch gemerged werden, müssen diese von einem anderen Teammitglied überprüft werden.
5. Schreibe Commit-Nachrichten auf Englisch.
6. Beim wöchentlichen Meeting werden die Änderungen vom Develop-Branch in den Main-Branch gemerged.

11.4 Dokumentation

Die Projektdokumentation wird vollständig mit Typst¹¹ erstellt. Die Entscheidung für Typst erfolgte aufgrund der einfacheren Syntax im Vergleich zu Latex¹², der nativen Git-Integration und der bereits im Vorprojekt begonnenen Typst-Vorlage.

11.4.1 Dokumentation Guidelines

1. Dokumentiere auf Deutsch.
2. Dokumentiere mit Typst.
3. Verwende die während diesem Projekt entwickelte Vorlage.
4. Unterteile den Text in eine Typst-Datei pro grösseres Kapitel.
5. Benenne die einzelnen Typst Dateien in Englisch und verwende hierfür kebab-case.
6. Verwende aussagekräftige Titel, welche den Text gut unterteilen.
7. Verseehe alle nötigen Titel mit einer kurzen und klaren Referenz-Variable im snake_case.
8. Verwende eine saubere Formatierung, die dabei, hilft den Text gut lesen zu können.
9. Kennzeichne Referenzen aus externen Quellen als solche.
10. Schreibe für jede Abbildung und Tabelle eine Beschriftung.
11. Schreibe Referenz-Namen der Beschriftungen für Abbildungen und Tabellen im kebab-case.

11.4.2 Dokumentation Deployment

Im Rahmen dieser Arbeit wurde eine automatisierte Dokumentations-Pipeline implementiert, welche die kontinuierliche Erstellung und Veröffentlichung der Projektdokumentation gewährleistet. Ziel war es, den zuvor im Vorprojekt auf GitLab basierenden CI/CD-Prozess auf die GitHub-Infrastruktur zu migrieren und an die Anforderungen des neuen Projektsetups anzupassen (siehe Abschnitt 11.3).

Die Pipeline dient der automatischen Kompilierung und Veröffentlichung der Typst-Dokumentation, bestehend aus dem Hauptbericht und den Sitzungsprotokollen. Dabei werden sämtliche Schritte, vom Erstellen der PDF-Dateien bis zur Bereitstellung auf GitHub Pages, automatisiert ausgeführt. Da GitHub Pages für private Repositories in der kostenfreien Version nicht verfügbar ist, wurde eine Split-Repository-Strategie umgesetzt. Das private Repository (clanscore-doc) enthält den vollständigen Typst-Quellcode, während ein separates öffentliches Ziel-Repository (clanscore-doc-pub) ausschliesslich die generierten Ausgabedateien beherbergt.

¹⁰<https://github.com/ClansCore/clanscore-doc-pub>

¹¹<https://typst.app/>

¹²<https://www.latex-project.org/>

Die Pipeline wurde mittels GitHub Actions realisiert und befindet sich im privaten Repository. Der Workflow wird bei jedem Push auf den Main-Branch ausgelöst. Für die Authentifizierung und das Deployment wurde eine eigene GitHub App innerhalb der Organisation ClansCore erstellt. Diese App besitzt ausschliesslich die für das Deployment benötigten Berechtigungen auf das Ziel-Repository. Das App-Zugriffstoken wird zur Laufzeit über eine Action generiert und anschliessend verwendet, um den Inhalt des public/-Ordners in den Main-Branch des öffentlichen Repositories zu pushen.

Die GitHub Pages des Ziel-Repositories wurden so konfiguriert, dass der Main-Branch automatisch als statische Website veröffentlicht wird. Damit wird bei jeder Änderung im privaten Repository die Dokumentation neu erzeugt und unmittelbar online verfügbar gemacht.

Dieser Ansatz stellt eine klare und sichere CI/CD-Lösung dar. Der Quellcode und interne Inhalte bleiben privat, während die erzeugten Enddokumente öffentlich zugänglich sind. Durch die Verwendung einer GitHub App wird zudem auf persönliche Zugriffstokens verzichtet, was eine nachhaltige und wartungsarme Authentifizierung gewährleistet.

11.5 Code und Entwicklung

Der Code für dieses Projekt wird in der hier beschriebenen Form erstellt.

11.5.1 Code Guidelines

1. Schreibe TypeScript Variablen- und Funktionsnamen im camelCase.
2. Schreibe TypeScript Konstanten in Grossbuchstaben, wobei einzelne Wörter mit Unterstrichen aufgeteilt werden.
3. Schreibe HTML-Element-Bezeichner im kebab-case.
4. Gib Arrays einen Namen im Plural.
5. Schreibe kurze Funktionen mit wenigen Argumenten. Eine Funktion soll nur einen Zweck erfüllen.
6. Verwende let oder const anstelle von var.
7. Schreibe Vergleiche mit === oder !==.
8. Verwende bei Variablen-Zuweisungen Leerzeichen zwischen dem Gleichheitszeichen.
9. Schreibe bei Funktionsheadern die geschweifte Klammer auf die gleiche Zeile.
10. Schreibe Kommentare nur wo es nötig ist, der Code sollte selbsterklärend geschrieben werden.
11. Verwende für gleiche Konzepte die gleichen Wörter.
12. Überprüfe Änderungen vor dem pushen mit Prettier und ESLint.

11.5.2 Setup

Um den Coding-Style einzuhalten werden die Standardregeln von Prettier und ESLint für TypeScript verwendet. Zudem sind bei den Coding Projekten bestimmte Prozesse automatisiert, um die Entwicklung zu erleichtern.

11.5.3 Hosting

Um [ClansCore](#) dauerhaft online zu halten und beispielsweise auf Discord-Ereignisse zu reagieren oder zuverlässigen Zugriff auf das Admin-Dashboard zu gewährleisten, ist ein geeignetes Hosting erforderlich. Während dem [Vorprojekt](#) wurde der Discord-Bot auf einem Linux-Server des Vereins [EGSwiss](#) gehostet, durch manuelle Deployments über den Vereinsinternen Serveradmin. Da dieser Serveradmin zum Zeitpunkt der Arbeit anderweitig beschäftigt war sowie die Teammitglieder keinen direkten Zugriff auf den Server hatten, fiel die Entscheidung auf einen temporären virtuellen Server der Fachhochschule OST. Dieser läuft in einer stabilen Umgebung, ist kostenfrei und garantiert dem Team vollen Zugriff für die Einführung eines automatischen Deployments (siehe [Abschnitt 11.5.4](#)).

11.5.4 Code Deployment

Im [Vorprojekt](#) wurde das Deployment mit Docker manuell durch eine Drittperson auf einem dedizierten Linux-Server durchgeführt. Da ein manuelles Deployment fehleranfällig, langsam und ineffizient ist, wird ein automatisiertes Deployment umzusetzen ([AutoMQ-Contributors, 2025](#)).

Mit GitHub Actions kann das Deployment automatisiert und sichergestellt werden, dass bei einem Push auf den Main-Branch die Änderungen in die Produktivumgebung übernommen werden. Docker wird für das Containermanagement verwendet,

da es eine zuverlässige und plattformunabhängige Bereitstellung der Anwendung ermöglicht ([Docker-Contributors, 2025](#)) und das Team bereits Erfahrung mit Docker hat.

11.6 Programmiersprachen und Frameworks

ClansCore baut auf einem bereits im [Vorprojekt](#) entwickelten Discord-Bot auf. Dieser wurde in TypeScript programmiert und verwendet die Bibliothek Discord.js, welche als die am weitesten verbreitete und aktiv gepflegte Schnittstelle zur Discord API gilt ([Discord.js-Contributors, 2025a](#)). Die Bibliothek abstrahiert die Kommunikation über [REST-](#) und [WebSocket-Schnittstellen](#) und ermöglicht dadurch eine effiziente und stabile Implementierung der Bot-Funktionalitäten.

Discord.js wird fortlaufend weiterentwickelt und erhält regelmässig sicherheits- und funktionsbezogene Updates ([Discord.js-Contributors, 2025b](#)). Durch die positiven Erfahrungen aus dem Vorprojekt, den aktiven Community-Support und die hohe Kompatibilität mit TypeScript, wird Discord.js weiterhin als technologische Basis verwendet. Diese Entscheidung stellt sicher, dass bestehende Codebestandteile übernommen und nachhaltig weiterentwickelt werden können, ohne eine kostspielige Migration auf eine alternative Bibliothek durchführen zu müssen.

Für die Entwicklung des Admin-Dashboards standen verschiedene moderne Web-Technologien zur Auswahl. Da ein Framework den Entwicklungsprozess strukturiert, die Wiederverwendbarkeit erhöht sowie die Wartung erleichtert, wurde ein komponentenbasiertes Framework eingesetzt. Die Auswahl erfolgte anhand von Kriterien wie Verbreitung, Funktionsumfang, Zukunftssicherheit und vorhandene Teamerfahrung.

Web-Technologie	Beschreibung
React	Eine weit verbreitete JavaScript-Bibliothek für User Interfaces mit Millionen von Anwendern weltweit. React ist komponentenbasiert, kombiniert Logik und Markup in derselben Datei und profitiert von einer grossen Entwickler-Community. Die Flexibilität der Bibliothek ermöglicht unterschiedliche Architekturansätze, erfordert jedoch zusätzliche Tools für Routing, State-Management und Build-Prozesse. (React-Contributors, 2025)
Angular	Ein umfassendes Web-Framework, das eine vollständige Entwicklungsumgebung für skalierbare Web-Applikationen bietet. Angular wird von Google gepflegt, unterstützt TypeScript nativ und beinhaltet bereits Funktionen wie Routing, Dependency Injection und Formularverwaltung. Es ermöglicht eine klare Strukturierung und eine nachhaltige Code-Architektur. (Angular-Contributors, 2025)
Vue.js	Ein progressives JavaScript-Framework zur Entwicklung von User Interfaces. Vue kombiniert einfache Syntax mit hoher Flexibilität und eignet sich sowohl für kleine als auch komplexe Anwendungen. Es besitzt eine engagierte Community, wird jedoch primär von Einzelentwicklern und kleineren Teams gepflegt. (Vue.js-Contributors, 2025)

Tab. 57: Beschreibung der evaluierten Web-Technologien

Nach einer systematischen Bewertung wurde entschieden, das Admin-Dashboard mit Angular zu entwickeln. Diese Entscheidung beruht auf mehreren Faktoren:

- **TypeScript-Integration:** Angular ist vollständig in TypeScript implementiert, was eine einheitliche Codebasis zwischen Frontend und Backend ermöglicht. Dadurch können Typdefinitionen und Schnittstellen gemeinsam genutzt werden, was die Konsistenz und Fehlersicherheit erhöht.
- **Vollständige Framework-Struktur:** Im Gegensatz zu React, das primär eine UI-Bibliothek darstellt, bietet Angular ein integriertes Framework mit klaren Architekturvorgaben, standardisierten Projektstrukturen und eingebautem Tooling (z. B. Testing, Routing, Dependency Injection). Dies reduziert den Integrationsaufwand externer Bibliotheken und erhöht die Wartbarkeit.
- **Skalierbarkeit und Nachhaltigkeit:** Angular ist auf gross angelegte, modular aufgebaute Anwendungen ausgelegt und gewährleistet damit eine langfristig stabile Codebasis.
- **Teamkompetenz:** Beide Teammitglieder verfügten bereits über Erfahrung im Umgang mit Angular, was den Einarbeitungsaufwand erheblich reduzierte und einen produktiven Entwicklungsstart ermöglichte. Mit React und Vue.js liegen hingegen keine oder nur geringe praktische Erfahrungen vor.

Zusammenfassend wurde Angular aufgrund der klaren Struktur, nativen TypeScript-Unterstützung, hohen Skalierbarkeit und der bestehenden Teamexpertise als geeignetstes Framework für die Umsetzung des Admin-Dashboards bewertet. Diese

Wahl stellt sicher, dass das Dashboard langfristig wartbar bleibt und sich konsistent in die bestehende ClansCore-Systemarchitektur integriert.

11.7 Datenbank

Im Vorprojekt wurde eine MongoDB-Datenbank verwendet. MongoDB ist eine dokumentenorientierte NoSQL-Datenbank, die Daten in JSON-ähnlichen Dokumenten speichert ([MongoDB-Contributors, 2025f](#)). Eine zentrale Funktion von MongoDB ist die horizontale Skalierbarkeit, was insbesondere bei einer steigenden Nutzung des Bots einen entscheidenden Vorteil darstellt. Die Erweiterungen, welche für ClansCore umgesetzt wurden, erforderten keine fundamentalen Änderungen an der Datenbank, sodass kein Anlass bestand, eine alternative Datenbanklösung in Erwägung zu ziehen.

11.8 KI-Tools

KI-Tools wurden eingesetzt, um die Effizienz zu steigern und die Qualität der Ergebnisse zu verbessern. Die Einsatzbereiche umfassen:

- **Technische Entscheidung:** Unterstützung bei der Bewertung von Frameworks, Bibliotheken und Architekturansätzen.
- **Programmierung:** Hilfe bei der Fehlersuche, Optimierung und Codegenerierung.
- **Dokumentation:** Unterstützung beim Strukturieren und Formulieren von Textabschnitten, insbesondere für technische Beschreibungen, Kapitelüberschriften und sprachliche Korrektheit.

Die KI wurde als Assistenzwerkzeug eingesetzt und diente nicht zur automatisierten Generierung vollständiger Inhalte. Alle durch die KI erzeugten Vorschläge wurden kritisch geprüft und bei Bedarf angepasst.

Folgende KI-Tools wurden verwendet:

- OpenAI ChatGPT¹³
- Google Gemini¹⁴
- GitHub Copilot¹⁵

¹³<https://chatgpt.com/>

¹⁴<https://gemini.google.com>

¹⁵<https://github.com/features/copilot>

12 Sitzungsprotokolle

Besprechung vom 19.09.2025

Anwesenheitsliste:	Frieder Loch, Joel Thoma, Vanessa Alves
Autor:	Joel Thoma

Inhalt der Sitzung:

- Aufgabenstellung
 - Was kommt als nächstes
 - Abgabetermine
 - Feedback SA FS25
 - Sonstiges
-

Aufgabenstellung

Ein konkretes Projekt finden. Bei anderen onlinebasierten Vereinen Umfragen und Recherche durchführen um Konzept zu verbessern.

Eine Idee ist ein Admin-Dashboard zu machen um von Discord zu entkoppeln und keine Datenbankzugriffe mehr nötig sind. Eine Anbindung an Microsoft Teams oder die Open Source Plattform Matrix wäre denkbar.

Was kommt als nächstes

Problembeschreibung, Zeitplan und Meilensteine definieren.

Abgabetermine

Wegen der SA von Joel ist der Abgabetermin für die Arbeit am 19. Dezember und Vanessa hat noch für die Präsentation und Poster bis am 9. Januar Zeit.

Feedback SA FS25

Wird noch bereitgestellt.

Sonstiges

Änderungen zur Aufgabenstellung:

Gegenleser: Severin Dellsperger

Koreferent: Julia Czerniak-Wilmes

Besprechung vom 29.09.2025

Anwesenheitsliste:	Frieder Loch, Joel Thoma, Vanessa Alves
Autor:	Joel Thoma

Inhalt der Sitzung:

- Besprechung aktueller Stand
 - ToDo
-

Besprechung aktueller Stand

- Meilensteine könnte man messbar machen (klar definieren wann diese erreicht sind)
- Im MVP sollte klar definiert sein welche Teile neu sind (Nur die Plattform-Unabhängigkeit in das MVP?)

ToDo

- Self-Determination-Theory: über dieses oder ähnliche Frameworks Research betreiben und in das Interview einbauen. (vorallem in Verbindung mit Competence, Autonomy, Relatedness)
- Problemstellung am Anfang sollte folgendes enthalten:
 - die zentralen technischen Problem finden und beschreiben. Wo könnte es Probleme geben und wieso ist es schwierig. (Kernprobleme, da unterschiedliche Lösungen finden und analysieren) (vlt Synchronisierung, Modularisierung?)
 - Alles Begründen. Alle Handlungsmöglichkeiten anschauen, analysieren und die Entscheidungen dokumentieren.
 - Das Produkt gut verkaufen. Was ist das endgültige Produkt. Gamification und Produkt gut zusammenpacken.

Besprechung vom 03.10.2025

Anwesenheitsliste:	Frieder Loch, Joel Thoma, Vanessa Alves
Autor:	Joel Thoma

Inhalt der Sitzung:

- Besprechung aktueller Stand
-

Besprechung aktueller Stand

- Es wurde der aktuelle Stand geschildert.
- Verlängerung der Elaboration Phase und Verschiebung vom Gamification Konzept

Besprechung vom 10.10.2025

Anwesenheitsliste:	Frieder Loch, Joel Thoma, Vanessa Alves
Autor:	Joel Thoma

Inhalt der Sitzung:

- GitHub Orga und CI/CD Doku
- Einleitung / Stand der Technik
- Anforderungen
- Interview andere Vereine
- Zwischenpräsentation

GitHub Orga und CI/CD Doku

Frieder wurde über die Organisation in GitHub informiert und die Möglichkeit Zugriff auf den Code und vor allem die online Version der aktuell im Main Branch deployte Dokumentation der Arbeit.

Organisation mit allen Repos: <https://github.com/ClansCore>

Einleitung / Stand der Technik

Wir haben zusammen die Einleitung und den Stand der Technik angeschaut und besprochen. Es ist nichts aufgefallen.

Anforderungen

Die Anforderungen (User Stories, FR, Use Cases und NFR) wurden angeschaut und besprochen.

Forlgende Anpassungen müssen noch gemacht werden:

- NFR3: Ist das ein Security Thema oder wie genau soll das gemessen werden? Evtl. Skript?
- NFR4: Welche UI-Regeln / Guidelines werden genau befolgt?
- NFR9: Messung muss genauer beschrieben werden

Interview andere Vereine

Die Interviews wurden durchgeführt und die unbearbeiteten Resultate vorgestellt. Diese werden diese Woche ausgewertet für das neue Gamification-Konzept.

Zwischenpräsentation

Vanessa wird voraussichtlich alleine die Präsentation Mitte Semester (Woche 7) durchführen. Joel darf freiwillig mitmachen oder zusehen.

Mögliche Daten für die Zwischenpräsentation, welche noch bestimmt wird.

- 28.10 - 15:00-17:00
- 29.10 - 13:00-15:00
- 30.10 - 13:00-15:00

13 Persönliche Erfahrungsberichte

13.1 Bericht Joel Thoma

13.2 Bericht Vanessa Alves

Kapitel III

Glossar und Abkürzungsverzeichnis

ClansCore	ClansCore ist das Gesamt-System und der Produktname für die Erweiterung des Discord-Bots aus dem <u>Vorprojekt</u> . Es ist das End-Produkt dieser Arbeit und verbindet sämtliche Teil-Komponenten wie Backend, Datenbank, Discord-Bot, Admin-Dashboard und weitere Plattformen.
EGSwiss	Ein Schweizer Gaming Verein, wobei das Teammitglied Vanessa Alves Präsidentin ist und daher die nötige Expertise für die Vorarbeit und aktuelle Weiterentwicklung mitbringt sowie Erfahrungen aus dem Vereinsalltag einfließen lässt.
CI/CD	Continuous Integration (CI) und Continuous Deployment/Delivery (CD). Ein automatisierter Prozess, bei dem Codeänderungen kontinuierlich integriert, getestet und automatisch oder mit manueller Freigabe in die Produktionsumgebung publiziert werden.
REST-Schnittstelle	REST (Representational State Transfer) ist eine Architekturform für webbasierte Schnittstellen, bei der Daten über standardisierte HTTP-Methoden (z. B. GET, POST, PATCH, DELETE) ausgetauscht werden. REST-APIs sind zustandslos, d. h. jede Anfrage enthält alle notwendigen Informationen, um sie unabhängig von früheren Interaktionen zu verarbeiten. In <u>ClansCore</u> wird die Discord REST-API verwendet, um gezielte Aktionen wie das Senden von Nachrichten oder das Verwalten von Rollen durchzuführen.
WebSocket-Schnittstelle	WebSockets sind bidirektionales Kommunikationsprotokolle (RFC 6455), welches eine dauerhafte Verbindung zwischen Client und Server ermöglicht. Im Gegensatz zu REST, das auf einzelnen HTTP-Anfragen basiert, erlaubt WebSocket eine kontinuierliche Übertragung von Daten in Echtzeit. In <u>ClansCore</u> wird über das Discord-Gateway eine WebSocket-Verbindung aufgebaut, um Ereignisse (z. B. neue Nachrichten, Interaktionen oder Mitgliederänderungen) unmittelbar zu empfangen und darauf zu reagieren.
Role-Based Access Control (RBAC)	Role-Based Access Control (RBAC) ist ein Sicherheitskonzept, bei dem der Zugriff auf Ressourcen anhand von Rollen vergeben wird. Benutzer erhalten dabei bestimmte Rollen, die jeweils unterschiedliche Berechtigungen für Funktionen und Daten gewähren. So können Administratoren granular steuern, wer auf welche Teile eines Systems zugreifen darf.
Zwei- / Multi-Faktor-Authentifizierung (2FA/MFA)	Zwei- oder Multi-Faktor-Authentifizierung (2FA/MFA) ist ein Sicherheitsmechanismus, bei dem zur Anmeldung an einem System zwei verschiedene Authentifizierungsfaktoren abgefragt werden. Dies können z. B. ein Passwort und ein einmaliger Code (z. B. aus einer App oder per SMS) sein. 2FA beziehungsweise MFA erhöht die Sicherheit erheblich, da ein Angreifer beide Faktoren kennen oder besitzen müsste, um sich erfolgreich anzumelden.
Virtual Private Network (VPN)	Ein Virtual Private Network (VPN) ist eine Technologie, die eine verschlüsselte Verbindung über ein öffentliches oder unsicheres Netzwerk ermöglicht. Durch die Nutzung eines VPNs wird der Datenverkehr zwischen Client und Server gesichert, wodurch Vertraulichkeit und Integrität der übertragenen Daten gewährleistet werden.

Bastion Host		Ein <u>gehärteter Server</u> , der als sicherer Zugangspunkt zu einem internen Netzwerk dient. Er vermittelt administrative Zugriffe über einen kontrollierten, <u>MFA</u> -geschützten Kanal und verhindert direkten Zugriff auf interne Systeme.
Time-based Password (TOTP)	One-Time	Ein zeitbasiertes Einmalpasswort ist ein Verfahren zur Zwei-Faktor-Authentifizierung (2FA), bei dem temporäre, nur wenige Sekunden gültige Passwörter generiert werden. Diese werden auf einem Authenticator-Gerät oder in einer App erstellt. Die Synchronisation erfolgt über eine gemeinsame geheime Basis und die aktuelle Zeit, was eine zusätzliche Sicherheitsebene beim Login-Prozess bietet.
Phishing		Phishing bezeichnet den Versuch, über gefälschte E-Mails, Websites oder Nachrichten vertrauliche Informationen wie Passwörter, Kreditkartendaten oder Zugangsdaten zu erlangen. Angreifer nutzen hierbei Social-Engineering-Methoden, um das Vertrauen von Nutzern zu erschleichen. Ziel ist meist der unbefugte Zugriff auf Konten oder Systeme. Schulungen und Sensibilisierung helfen, Phishing-Angriffe zu erkennen und zu vermeiden.
CLOUD Act		Der Clarifying Lawful Overseas Use of Data Act (CLOUD Act) ist ein US-amerikanisches Gesetz von 2018, das US-Behörden den Zugriff auf Daten ermöglicht, die von US-Unternehmen gespeichert werden, auch wenn sich diese Daten physisch ausserhalb der USA befinden. Für in der Schweiz gehostete Systeme bedeutet dies, dass bei Nutzung von US-Cloud-Anbietern theoretisch ein ausländischer Datenzugriff möglich ist. Daher wird häufig auf Hosting in der Schweiz gesetzt, um die nationale Datenhoheit zu gewährleisten.
Systemhärtung (Hardening)	(Har-	Massnahmen zur Reduktion der Angriffsfläche eines Systems durch Entfernen unnötiger Dienste, Einschränkung von Benutzerrechten und konsequente Sicherheitskonfiguration. Ziel ist es, die Widerstandsfähigkeit gegen Angriffe zu erhöhen und unautorisierte Zugriffe zu verhindern.

Kapitel IV

Literaturverzeichnis

- Alves, V., & Schnider, M. (2025). *Discord-Bot für Vereine*.
- Angular-Contributors. (2025,). *What is Angular?*. <https://angular.dev/overview>
- Arcane-Contributors. (2025,). *Arcane Discord Bot*. <https://arcane.bot/>
- AutoMQ-Contributors. (2025,). *Common Issues with Manual Deployment*. <https://www.automq.com/blog/fully-automated-deployment-vs-manual-deployment-comparison#common-issues-with-manual-deployment>
- Chou, Y.-k. (2015). *Actionable Gamification: Beyond Points, Badges and Leaderboards* (, Hrsg.). CreateSpace Independent Publishing Platform.
- CISA. (2022). *Implementing Multi-Factor Authentication (MFA) for Remote Access and VPNs*. https://www.cisa.gov/sites/default/files/publications/CISA_MFA_Guidelines_Remote_Access_2022.pdf
- ClubDesk-Contributors. (2025,). *Vereinssoftware für die einfache Vereinsverwaltung*. <https://www.clubdesk.de/de/startseite>
- Discord.js-Contributors. (2025b,). *Commits*. <https://github.com/discordjs/discord.js/commits/main/>
- Discord.js-Contributors. (2025a,). *The most popular way to build Discord bots*. <https://discord.js.org/>
- Docker-Contributors. (2025,). *Developers bring their ideas to life with Docker*. <https://www.docker.com/why-docker/>
- Europäische-Union. (2016,). *VERORDNUNG (EU) 2016/679 DES EUROPÄISCHEN PARLAMENTS UND DES RATES*. <https://eur-lex.europa.eu/legal-content/DE/TXT/?uri=CELEX:32016R0679>
- Martin, F. (2003,). *Optimistic Offline Lock*. <https://martinfowler.com/eaCatalog/optimisticOfflineLock.html>
- MEE6-Contributors. (2025,). *Statistics Channels Plugin*. <https://wiki.mee6.xyz/en/plugins/statistics-channels>
- Microsoft-Contributors. (2025b,). *Planning for mandatory multifactor authentication for Azure and other admin portals*. <https://learn.microsoft.com/en-us/entra/identity/authentication/concept-mandatory-multifactor-authentication?tabs=dotnet>
- Microsoft-Contributors. (2025a,). *Security at your organization: Multifactor authentication statistics*. <https://learn.microsoft.com/en-us/partner-center/security/security-at-your-organization>
- MongoDB-Contributors. (2025b,). *Authentication and Authorization with OIDC/OAuth 2.0*. <https://www.mongodb.com/docs/manual/core/oidc/security-oidc/>
- MongoDB-Contributors. (2025f,). *Introduction to MongoDB*. <https://www.mongodb.com/docs/manual/introduction/>
- MongoDB-Contributors. (2025d,). *Role-Based Access Control in Self-Managed Deployments*. <https://www.mongodb.com/docs/manual/core/authorization/>
- MongoDB-Contributors. (2025c,). *SCRAM*. <https://www.mongodb.com/docs/manual/core/security-scram/>
- MongoDB-Contributors. (2025e,). *Transactions*. <https://www.mongodb.com/docs/manual/core/transactions/>
- MongoDB-Contributors. (2025a,). *Use X.509 Certificates to Authenticate Clients on Self-Managed Deployments*. <https://www.mongodb.com/docs/manual/tutorial/configure-x509-client-authentication/>
- Murugiah, S., & Karen, S. (2017). *Application Container Security Guide (NIST SP 800-190)*. <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-190.pdf>
- NIST. (2020). *Security and Privacy Controls for Information Systems and Organizations (SP 800-53 Rev. 5)*. <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-53r5.pdf>
- OWASP-Cheat-Sheets-Series-Contributors. (2025b,). *Cross-Site Request Forgery Prevention Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- OWASP-Cheat-Sheets-Series-Contributors. (2025a,). *Session Management Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

OWASP-Contributors. (2021,). *OWASP Top 10*. <https://owasp.org/Top10/>

React-Contributors. (2025,). *React*. <https://react.dev/>

Schweizerische Eidgenossenschaft. (2020,). *Bundesgesetz über den Datenschutz (Datenschutzgesetz, DSG)*. <https://www.fedlex.admin.ch/eli/cc/2022/491/de>

U.S.-Department-of-Justice. (2019,). *The Purpose and Impact of the CLOUD Act - FAQs*. <https://www.justice.gov/criminal/media/999616/dl?inline>

Vue.js-Contributors. (2025,). *What is Vue?*. <https://vuejs.org/guide/introduction>

WildApricot-Community. (2015,). *Gamify Member's Site Participation*. <https://forums.wildapricot.com/forums/308932-wishlist/suggestions/10321731-gamify-member-s-site-participation>

WildApricot-Contributors. (2025,). *Membership Management Software*. <https://www.wildapricot.com/>

Kapitel V

Abbildungsverzeichnis

Abb. 1 ClansCore im Octalysis Modell	17
Abb. 2 Login	25
Abb. 3 Bewerbungsformular	25
Abb. 4 Ranglisten-Seite	25
Abb. 5 Aufgaben-Seite	26
Abb. 6 Mitglieder-Seite	26

Kapitel VI

Tabellenverzeichnis

Tab. 1	Beschreibung der Prioritäts-Kategorien	4
Tab. 2	Beschreibung der Resultats-Kategorien	4
Tab. 3	Beschreibung Functional Requirement 1	5
Tab. 4	Beschreibung Functional Requirement 2	5
Tab. 5	Beschreibung Functional Requirement 3	5
Tab. 6	Beschreibung Functional Requirement 4	6
Tab. 7	Beschreibung Functional Requirement 5	6
Tab. 8	Beschreibung Functional Requirement 6	6
Tab. 9	Beschreibung Functional Requirement 7	6
Tab. 10	Beschreibung Functional Requirement 8	6
Tab. 11	Beschreibung Functional Requirement 9	6
Tab. 12	Beschreibung Functional Requirement 10	7
Tab. 13	Beschreibung Functional Requirement 11	7
Tab. 14	Beschreibung Functional Requirement 12	7
Tab. 15	Beschreibung Fully dressed Use Case 1	7
Tab. 16	Beschreibung Fully dressed Use Case 2	8
Tab. 17	Beschreibung Fully dressed Use Case 3	8
Tab. 18	Beschreibung Fully dressed Use Case 4	8
Tab. 19	Beschreibung Fully dressed Use Case 5	9
Tab. 20	Beschreibung Fully dressed Use Case 6	9
Tab. 21	Beschreibung Fully dressed Use Case 7	9
Tab. 22	Beschreibung Fully dressed Use Case 8	10
Tab. 23	Beschreibung Fully dressed Use Case 9	10
Tab. 24	Beschreibung Fully dressed Use Case 10	11
Tab. 25	Beschreibung Fully dressed Use Case 11	11

Tab. 26 Beschreibung Non-Functional Requirement 1	12
Tab. 27 Beschreibung Non-Functional Requirement 2	12
Tab. 28 Beschreibung Non-Functional Requirement 3	12
Tab. 29 Beschreibung Non-Functional Requirement 4	12
Tab. 30 Beschreibung Non-Functional Requirement 5	13
Tab. 31 Beschreibung Non-Functional Requirement 6	13
Tab. 32 Beschreibung Non-Functional Requirement 7	13
Tab. 33 Beschreibung Non-Functional Requirement 8	13
Tab. 34 Beschreibung Non-Functional Requirement 9	14
Tab. 35 Beschreibung Non-Functional Requirement 10	14
Tab. 36 Beschreibung der vier grundlegenden Prinzipien	19
Tab. 37 Varianten zur Absicherung des DB-Zugangs	21
Tab. 38 Beschreibung der abgesicherten Kanäle in ClansCore	22
Tab. 39 Einschätzung der Risiken und deren Massnahmen	24
Tab. 40 Long-Term Plan	31
Tab. 41 Risiko Matrix	33
Tab. 42 Beschreibung Risiko 1	33
Tab. 43 Beschreibung Risiko 2	34
Tab. 44 Beschreibung Risiko 3	34
Tab. 45 Beschreibung Risiko 4	34
Tab. 46 Beschreibung Risiko 5	34
Tab. 47 Beschreibung Risiko 6	34
Tab. 48 Beschreibung Risiko 7	35
Tab. 49 Beschreibung Risiko 8	35
Tab. 50 Beschreibung Risiko 9	35
Tab. 51 Beschreibung Risiko 10	35
Tab. 52 Beschreibung Risiko 11	35
Tab. 53 Beschreibung Risiko 12	35
Tab. 54 Beschreibung Risiko 13	36

Tab. 55 Beschreibung Risiko 14	36
Tab. 56 Übersicht der Repositories innerhalb der GitHub-Organisation ClansCore	38
Tab. 57 Beschreibung der evaluierten Web-Technologien	41

Kapitel VII

Code-Listings

Kapitel VIII

Anhang